

อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์

Elderly fall detection device with embedded artificial intelligence on a microcontroller

ธนภัทร เลิศพิทักษ์สิทธิ์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตร์อุตสาหกรรมบัณฑิต
สาขาวิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น
พ.ศ. 2569
ลิขสิทธิ์ของคณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน

อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์

ธนภัทร เลิศพิทักษ์สิทธิ์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตร์อุตสาหกรรม
บัณฑิต
สาขาวิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น
พ.ศ. 2569
ลิขสิทธิ์ของคณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน

Elderly fall detection device with embedded artificial intelligence on a microcontroller

Thanapat Lertpitaksit

A Report Submitted in Partial Fulfillment of the Requirements for
the Degree of Bachelor of Engineering
Department of Computer Engineering, Faculty of Engineering
Rajamangala University of Technology Isan, Khon Kaen Campus
2023

© Faculty of Engineering, Rajamangala University of Technology Isan

หัวข้อปริญญานิพนธ์ : อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์
จัดทำโดย : ธนภัทร เลิศพิทักษ์สิทธิ์
สาขาวิชา : วิศวกรรมคอมพิวเตอร์
ประธานที่ปรึกษา : อาจารย์ ดอกเตอร์ประภาส ผ่องสนาม

ได้รับอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาครุศาสตร์อุตสาหกรรมบัณฑิต คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น

.....คณบดีคณะครุศาสตร์อุตสาหกรรม
(.....) วันที่.....

คณะกรรมการสอบปริญญานิพนธ์

.....ประธานกรรมการสอบ .....ประธานที่ปรึกษา
(.....) (อาจารย์ ดอกเตอร์ประภาส ผ่องสนาม)

.....กรรมการสอบ
(.....)

.....กรรมการสอบ
(.....)

หัวข้อปริญญานิพนธ์	อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์
จัดทำโดย	ธนภัทร เลิศพิทักษ์สิทธิ์
ปีที่ปริญญานิพนธ์สำเร็จ	พ.ศ. 2569
สาขาวิชา	วิศวกรรมคอมพิวเตอร์
ประธานที่ปรึกษา	อาจารย์ ดอกเตอร์ประภาส ผ่องสนาม

บทคัดย่อ

จุดประสงค์ เพื่อพัฒนาอุปกรณ์ตรวจจับการล้มต้นทุนต่ำที่ใช้เทคโนโลยีปัญญาประดิษฐ์ฝังตัว เพื่อสร้างและฝึกโมเดล TinyML สำหรับจำแนกเหตุการณ์การล้มจากข้อมูลเซนเซอร์ IMU เพื่อออกแบบและนำกลยุทธ์ Hybrid database มาใช้ในการจัดการข้อมูลที่มีโครงสร้างสูง (SQL) และข้อมูลแบบ Real-time (NoSQL) อย่างมีประสิทธิภาพ และเพื่อประเมินความแม่นยำและความรวดเร็วของการตรวจจับแบบเรียลไทม์บนไมโครคอนโทรลเลอร์

วิธีการดำเนินงาน การดำเนินงานเริ่มต้นด้วยการออกแบบและสร้างอุปกรณ์ฮาร์ดแวรต้นแบบโดยใช้ไมโครคอนโทรลเลอร์ ESP32 เชื่อมต่อกับเซนเซอร์ IMU (MPU6050) สำหรับตรวจจับความเร่งและอัตราการหมุน ร่วมกับโมดูล GPS (NEO-7M) สำหรับระบุพิกัดตำแหน่ง จากนั้นทำการเก็บข้อมูลการเคลื่อนไหวและการจำลองเหตุการณ์ล้มจากอาสาสมัครเพื่อนำไปฝึกฝนโมเดลการเรียนรู้ของเครื่อง (Machine Learning) แล้วแปลงเป็นโมเดลปัญญาประดิษฐ์ขนาดเล็ก (TinyML) ด้วย TensorFlow Lite for Microcontrollers เพื่อนำไปติดตั้งและประประมวลผลบนบอร์ด ESP32 โดยตรงแบบออฟไลน์ (On-device Processing) นอกจากนี้ได้พัฒนาระบบเว็บแอปพลิเคชัน (React และ Node.js) ร่วมกับกลยุทธ์ Hybrid Database โดยใช้ SQL (MySQL/PostgreSQL ร่วมกับ Prisma ORM) ในการจัดเก็บข้อมูลผู้ใช้และบันทึกเหตุการณ์ล้มถาวร (Event Logs) และใช้ NoSQL (Firebase/Supabase) สำหรับการรับส่งข้อมูลแชทและอัปเดตสถานะอุปกรณ์แบบ Real-time

ผลการดำเนินงาน จากการพัฒนาและทดสอบระบบ พบว่าอุปกรณ์สามารถประมวลผลสัญญาณจากเซนเซอร์ IMU และจำแนกเหตุการณ์การล้มออกจากกิจกรรมในชีวิตประจำวัน (ADL) ได้อย่างมีประสิทธิภาพบนตัวอุปกรณ์แบบออฟไลน์ ระบบมีความแม่นยำ (Accuracy) และความไว (Recall) สูง รวมถึงมีระยะเวลาตอบสนอง (Latency) ที่ต่ำ เมื่อเกิดเหตุการณ์ล้ม อุปกรณ์สามารถส่งสัญญาณแจ้งเตือนในพื้นที่ผ่าน Buzzer และไฟ LED พร้อมทั้งส่งข้อมูลตำแหน่งและอัปเดตสถานะไปยังเว็บแอปพลิเคชันผ่านฐานข้อมูล NoSQL แบบ Real-time ได้ทันที ตลอดจนสามารถบันทึกประวัติเหตุการณ์ลงในฐานข้อมูล SQL ได้อย่างถูกต้องเป็นระบบ ซึ่งช่วยลดระยะเวลาในการเข้าช่วยเหลือผู้สูงอายุและลดความเสี่ยงจากการนอนบาดเจ็บเป็นเวลานานได้อย่างมีประสิทธิผล

คำสำคัญ : ปัญญาประดิษฐ์ฝังตัว, การตรวจจับการล้ม, ไทนีเอ็มเอล, ไมโครคอนโทรลเลอร์ ESP32, ฐานข้อมูลแบบผสมผสาน

Project Title	Elderly fall detection device with embedded artificial intelligence on a microcontroller
Proposed by	Thanapat Lertphitaksit
Year	2026
Department	Computer Engineering
Advisor	Dr. Prapas Phongnam

Abstract

Objectives: To develop a low-cost fall detection device using embedded artificial intelligence technology; to create and train a TinyML model for classifying fall events based on IMU sensor data; to design and implement a hybrid database strategy for efficiently managing structured data (SQL) and real-time data (NoSQL); and to evaluate the real-time accuracy and responsiveness of the detection system on a microcontroller.

Methods: The development began with designing and prototyping the hardware using an ESP32 microcontroller connected to an IMU sensor (MPU6050) for measuring acceleration and angular rate, along with a GPS module (NEO-7M) for position tracking. Motion data and simulated fall events were collected from volunteers to train machine learning models. The trained model was then converted into a compact embedded AI model (TinyML) using TensorFlow Lite for Microcontrollers, enabling direct, offline, on-device processing on the ESP32 board. Additionally, a web application was developed using React and Node.js integrated with a hybrid database architecture. SQL (MySQL/PostgreSQL with Prisma ORM) was used for storing structured user profiles and permanent event logs, while NoSQL (Firebase/Supabase) handled chat sessions and real-time device status updates.

Results: The system evaluation demonstrated that the device successfully processed IMU sensor signals and effectively classified fall events from activities of daily living (ADL) directly on the device offline. The system achieved high accuracy and recall with low response latency. Upon detecting a fall, the device triggered local alerts via a buzzer and LED indicators, while simultaneously transmitting location data and updating device status to the web application in real time via the NoSQL database. Furthermore, event history was systematically recorded in the SQL database, effectively reducing emergency response time and minimizing risks associated with prolonged post-fall distress.

Keywords : Embedded AI, Fall Detection, TinyML, ESP32 Microcontroller, Hybrid Database

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีด้วยความกรุณาและความช่วยเหลือเป็นอย่างยิ่งจาก อาจารย์ ดร.ประภาส ผ่องสนาม อาจารย์ที่ปรึกษาปริญญานิพนธ์ ที่ได้ให้คำแนะนำ ข้อคิดเห็น ตรวจสอบแก้ไขข้อบกพร่องต่าง ๆ ด้วยความเอาใจใส่และถ่ายทอดองค์ความรู้ตลอดการดำเนินงาน ตลอดจนขอขอบพระคุณ อาจารย์ ดร.ชัย วันคัน ประธานกรรมการสอบ และ อาจารย์อรินธร เจษฎาเมธาขจร กรรมการสอบ ที่ได้กรุณาให้คำแนะนำและข้อเสนอแนะที่เป็นประโยชน์ยิ่ง ทำให้ปริญญานิพนธ์ฉบับนี้มีความสมบูรณ์ยิ่งขึ้น ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

ขอขอบพระคุณคณาจารย์และเจ้าหน้าที่สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น ทุกท่าน ที่ได้ประสิทธิ์ประสาทวิชาความรู้ อำนวยความสะดวกด้านสถานที่ อุปกรณ์ และเครื่องมือในห้องปฏิบัติการสำหรับการทดสอบระบบ ตลอดจนการสนับสนุนด้านการดำเนินงานต่าง ๆ ทำให้การพัฒนาระบบสามารถดำเนินไปได้อย่างเรียบร้อยและมีประสิทธิภาพ

นอกจากนี้ ขอขอบพระคุณกลุ่มอาสาสมัครทุกท่านที่ให้ความร่วมมือในการทดสอบและเก็บข้อมูลการเคลื่อนไหวเพื่อนำมาฝึกฝนโมเดลปัญญาประดิษฐ์ ตลอดจนขอขอบคุณเพื่อน ๆ นักศึกษาสาขาวิชาวิศวกรรมคอมพิวเตอร์ที่ได้คอยช่วยเหลือ แลกเปลี่ยนความรู้ และให้กำลังใจซึ่งกันและกันเสมอมา

ประโยชน์และคุณค่าอันพึงมีจากปริญญานิพนธ์ฉบับนี้ ผู้จัดทำโครงงานขอมอบเป็นกตัญญุตาบูชาแด่ บิดามารดา ครูอาจารย์ตลอดจนผู้มีพระคุณทุกท่าน

ธนภัทร เลิศพิทักษ์สิทธิ์

สารบัญ

เรื่อง	หน้า
บทคัดย่อภาษาไทย	ข
Abstract (บทคัดย่อภาษาอังกฤษ)	ค
กิตติกรรมประกาศ	จ
สารบัญ	ฉ
สารบัญตาราง	ช
สารบัญรูป	ซ
คำอธิบายสัญลักษณ์และคำย่อ	ญ
บทที่ 1 บทนำ	**1**
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงงาน	2
1.3 สมมติฐาน	2
1.4 ข้อตกลงเบื้องต้น	2
1.5 ขอบเขตของโครงงาน	4
1.6 ตัวแปรที่ศึกษา	4
1.7 ประโยชน์ที่คาดว่าจะได้รับ	6
1.8 คำนิยามศัพท์	7
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	**9**
2.1 สรีรวิทยาและพลศาสตร์ของการล้มในผู้สูงอายุ	10
2.2 เซ็นเซอร์วัดการเคลื่อนไหว (Inertial Measurement Unit: IMU)	10

2.3 ไมโครคอนโทรลเลอร์และการประมวลผลฝังตัว ESP32 10

2.4 ปัญญาประดิษฐ์ขนาดเล็กและการประมวลผลบนอุปกรณ์ขอบ (TinyML / Edge AI) 11

2.5 การระบุตำแหน่งภายในอาคารและการนับก้าว (Indoor Localization & Step Counting) 12

2.6 ระบบเครือข่ายและสถาปัตยกรรมเว็บแอปพลิเคชัน 13

2.7 งานวิจัยที่เกี่ยวข้อง (Literature Review) 14

บทที่ 3 วิธีดำเนินงาน 15

3.1 สถาปัตยกรรมระบบโดยรวม (System Architecture) 15

3.2 การออกแบบและพัฒนาฮาร์ดแวร์ตรวจจับการล้ม 16

3.3 การเตรียมชุดข้อมูลและการประมวลผลสัญญาณ (Dataset Preparation & Signal Preprocessing) 17

3.4 การสร้างและฝึกสอนโมเดลปัญญาประดิษฐ์ TinyML (TinyML Model Training) 19

3.5 การพัฒนาเฟิร์มแวร์ C++ Forward Pass บน ESP32 (Firmware Implementation) 20

3.6 การพัฒนาระบบคลาวด์และเว็บแอปพลิเคชัน (Cloud & Web Dashboard Implementation) 21

3.7 ขั้นตอนการทดสอบและประเมินผลประสิทธิภาพระบบ (Testing & Evaluation Methodology) 22

บทที่ 4 ผลการดำเนินงาน 24

4.1 ผลการประเมินประสิทธิภาพของโมเดล TinyML (TinyML Model Evaluation) 24

4.2 ผลการทดสอบการตรวจจับการล้มบนอุปกรณ์ฮาร์ดแวร์จริง(Hardware In-the-Loop Testing) 25

4.3 ผลการวัดประสิทธิภาพทรัพยากรบนไมโครคอนโทรลเลอร์ ESP32 (Resource Utilization) 26

4.4 ผลการทดสอบระบบการสื่อสารและแจ้งเตือนฉุกเฉินแบบเรียลไทม์ (Notification Latency & Web Dashboard) 27

บทที่ 5 สรุปผล และข้อเสนอแนะ 29
5.1 สรุปผลการวิจัย (Conclusion) 29

5.2 อภิปรายผลการวิจัย (Discussion) 30

5.3 ข้อเสนอแนะสำหรับการวิจัยและพัฒนาในอนาคต (Recommendations for Future Work) 30

บรรณานุกรม 32
ภาคผนวก 34
ภาคผนวก ก การจัดเตรียมสภาพแวดล้อมและไลบรารี (Environment Setup) 35
ภาคผนวก ข โครงสร้างโฟลเดอร์ชุดข้อมูล (Datasets Structure) 52
ภาคผนวก ค คำสั่งรันการเทรนโมเดลทีละขั้นตอน (Execution Commands) 61
ภาคผนวก ง รันโมเดลทีละขั้นตอน (Execution Commands) 65
ภาคผนวก จ ขั้นตอนการใช้งาน อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ 67

ประวัติผู้เขียน 78

สารบัญตาราง

เรื่อง หน้า

สารบัญรูป

เรื่อง หน้า

คำอธิบายสัญลักษณ์และคำย่อ

สัญลักษณ์และคำย่อ	ความหมาย
TinyML	TinyMachineLearning(การประมวลผลการเรียนรู้ของเครื่องบนอุปกรณ์ขนาดเล็ก)
IMU	Inertial Measurement Unit (เซ็นเซอร์วัดหน่วยความเฉื่อย 6 แกน)
ESP32	ไมโครคอนโทรลเลอร์ 32-บิต พร้อมชิปสื่อสาร Wi-Fi และ Bluetooth ในตัว
MPU-6050	โมดูลเซ็นเซอร์ตรวจวัดความเร่ง 3 แกน และความเร็วเชิงมุม 3 แกน
MLP	Multi-Layer Perceptron (โครงข่ายประสาทเทียมแบบหลายชั้น)
ReLU	Rectified Linear Unit (ฟังก์ชันกระตุ้นแบบเชิงเส้นตรงแบบปรับแก้)
Sigmoid	ฟังก์ชันกระตุ้นรูปตัวเอสสำหรับการคำนวณความน่าจะเป็นในช่วง 0 ถึง 1
SVM	Signal Vector Magnitude (ขนาดเวกเตอร์ความเร่งลัพธ์รวม)
SMA	Signal Magnitude Area (พื้นที่ขนาดสัญญาณความเร่งรวม)
Jerk	อัตราการเปลี่ยนแปลงความเร่งต่อหน่วยเวลา (อนุพันธ์อันดับหนึ่งของความเร่ง)
RSSI	Received Signal Strength Indication (ดัชนีชี้วัดความแรงของสัญญาณที่ได้รับ)
ADL	Activities of Daily Living (กิจกรรมการเคลื่อนไหวในชีวิตประจำวันปกติ)
Free-Fall	สภาวะการเคลื่อนที่ตกอย่างอิสระไร้น้ำหนัก (ค่าความเร่งเข้าใกล้ 0g)
Impact	แรงกระแทกจากการชนหรือกระทบกับพื้นผิว (ค่าความเร่งพุ่งสูง > 3.5g)
Forward Pass	การคำนวณส่งต่อข้อมูลไปข้างหน้าในโครงข่ายประสาทเทียม
Kalman Filter	อัลกอริทึมการกรองสัญญาณเพื่อประมาณค่าทางสถิติและลดสัญญาณรบกวน
Socket.IO	ไลบรารีสำหรับการรับส่งข้อมูลแบบสองทิศทางตามเวลาจริง(Real-time WebSocket)
REST API	Representational State Transfer Application Programming Interface
JSON	JavaScript Object Notation (รูปแบบการแลกเปลี่ยนข้อมูลเชิงโครงสร้าง)
NoSQL	ระบบจัดการฐานข้อมูลแบบไม่มีโครงสร้างเชิงสัมพันธ์ (เช่น MongoDB)
RDBMS	Relational Database Management System (เช่น MariaDB / MySQL)
Gemini AI	แบบจำลองภาษาขนาดใหญ่ (LLM) ของ Google สำหรับการวิเคราะห์เชิงลึก
Telegram Bot	ระบบอินเทอร์เฟซโปรแกรมประยุกต์สำหรับการส่งข้อความอัตโนมัติของ Telegram
BLE	Bluetooth Low Energy (บลูทูธพลังงานต่ำ)
I2C	Inter-Integrated Circuit (โปรโตคอลการสื่อสารข้อมูลแบบอนุกรมสองสาย)
UART	UniversalAsynchronousReceiverTransmitter(การสื่อสารอนุกรม

แบบอะซิงโครนัส)

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบันโครงสร้างประชากรทั่วโลกกำลังก้าวเข้าสู่สังคมผู้สูงอายุอย่างสมบูรณ์ (Aged Society) โดยเฉพาะในประเทศไทยที่มีสัดส่วนประชากรผู้สูงอายุที่มีอายุตั้งแต่60ปีขึ้นไปเพิ่มสูงขึ้นอย่างต่อเนื่อง ข้อมูลจากองค์การอนามัยโลก (WorldHealthOrganization:WHO) ระบุว่าอุบัติเหตุการหกล้มเป็นสาเหตุอันดับต้น ๆ ของการเสียชีวิตจากการบาดเจ็บโดยไม่เจตนาในผู้สูงอายุ และพบว่าผู้สูงอายุที่มีอายุตั้งแต่ 65 ปีขึ้นไปจะมีอัตราการหกล้มอย่างน้อยปีละ 1 ครั้งคิดเป็นร้อยละ 28 ถึง 35 ของประชากรผู้สูงอายุทั้งหมด ซึ่งผลกระทบจากการหกล้มไม่เพียงแต่นำมาซึ่งการบาดเจ็บทางกายภาพ เช่น กระดูกสะโพกหัก กระดูกสันหลังยุบตัว หรือการเกิดภาวะเลือดออกในกะโหลกศีรษะเท่านั้น แต่ยังส่งผลกระทบทางจิตวิทยาอย่างรุนแรง เช่น ภาวะกลัวการหกล้มซ้ำ (Fear of Falling) นำไปสู่การจำกัดการเคลื่อนไหว กล้ามเนื้อลีบ และสูญเสียความสามารถในการพึ่งพาตนเองในท้ายที่สุด

ปัญหาที่สำคัญอย่างยิ่งซึ่งส่งผลต่ออัตราการรอดชีวิตและความพิการของผู้สูงอายุเมื่อเกิดเหตุการณ์หกล้ม คือ "ภาวะการนอนค้างที่พื้นเป็นเวลานาน" (Long Lie Syndrome) ซึ่งเกิดขึ้นเมื่อผู้สูงอายุหกล้มในขณะที่อยู่ตามลำพังและไม่สามารถลุกขึ้นช่วยเหลือตนเองหรือส่งสัญญาณขอความช่วยเหลือได้ การนอนอยู่บนพื้นเป็นเวลานานเกินกว่า 1 ชั่วโมงก่อให้เกิดภาวะแทรกซ้อนที่อันตรายถึงชีวิต เช่น ภาวะกล้ามเนื้อลายสลาย (Rhabdomyolysis) แผลกดทับรุนแรง ภาวะขาดน้ำ และภาวะอุณหภูมิกายต่ำ (Hypothermia) ซึ่งส่งผลให้อัตราการเสียชีวิตภายใน 6 เดือนหลังเกิดเหตุสูงขึ้นถึงร้อยละ 50 ดังนั้น การมีระบบตรวจจับการหกล้มที่สามารถตรวจพบเหตุการณ์ได้อย่างแม่นยำ รวดเร็ว และส่งสัญญาณแจ้งเตือนไปยังผู้ดูแลหรือบุคลากรทางการแพทย์ได้ในทันทีจึงเป็นปัจจัยสำคัญยิ่งในการรักษาชีวิตและลดความรุนแรงของผลกระทบ

ที่ผ่านมาได้มีการพัฒนาระบบตรวจจับการล้มหลากหลายรูปแบบ ซึ่งสามารถจำแนกออกเป็น 3 กลุ่มหลัก ได้แก่ ระบบตรวจจับด้วยกล้องและภาพคอมพิวเตอร์วิทัศน์ (Vision-based Systems), ระบบตรวจจับด้วยสัญญาณคลื่นวิทยุหรือเซ็นเซอร์ติดตั้งรอบห้อง (Ambient/RF-based Systems), และระบบตรวจจับด้วยอุปกรณ์สวมใส่ (Wearable-based Systems) อย่างไรก็ตาม ระบบตรวจจับด้วยกล้องมีข้อจำกัดด้านความเป็นส่วนตัว (Privacy Issues) ของผู้ใช้งาน โดยเฉพาะอย่างยิ่งในพื้นที่ส่วนตัว เช่น ห้องนอนและห้องน้ำ ซึ่งเป็นจุดที่เกิดเหตุการณ์หกล้มบ่อยที่สุด อีกทั้งยังมีข้อจำกัดเรื่องมุมอับสายตา (Blind Spots) และสภาพแสงสว่าง ส่วนระบบเซ็นเซอร์ติดตั้งรอบห้องมีต้นทุนการติดตั้งที่สูงและจำกัดพื้นที่ใช้งาน อุปกรณ์ตรวจจับการล้มแบบสวมใส่จึงเป็นแนวทางที่ได้รับความนิยมสูงสุดเนื่องจากสามารถติดตามผู้ใช้งานได้ตลอดเวลาและมีความเป็นส่วนตัวสูง

อย่างไรก็ดี อุปกรณ์ตรวจจับการล้มแบบสวมใส่ในยุคแรกมักอาศัยการตั้งค่าเกณฑ์คงที่ (Threshold-based Methods) จากเซ็นเซอร์วัดความเร่ง ซึ่งมีข้อจำกัดอย่างยิ่งในการแยกแยะความแตกต่างระหว่างเหตุการณ์หกล้มจริงกับกิจกรรมปกติในชีวิตประจำวัน (Activities of Daily Living: ADL) ที่มีแรงกระแทกหรือการเปลี่ยนระดับความสูงอย่างรวดเร็ว เช่น การทิ้งตัวลงนั่งบนเก้าอี้ การล้มตัวลงนอน หรือการก้มเก็บของ ส่งผลให้อัตราการแจ้งเตือนผิดพลาด (False Positive / False Alarm) สูงมาก ทำให้ผู้ใช้งานเกิดความรำคาญและเลิกใช้งานอุปกรณ์ในที่สุด ขณะเดียวกันการส่งข้อมูลสัญญาณดิบขึ้นไปประมวลผลบนระบบคลาวด์ (Cloud-based AI) ต้องเผชิญกับปัญหาการสิ้นเปลืองพลังงานแบตเตอรี่ ความล่าช้าในการรับส่งข้อมูล (Network Latency) และไม่สามารถทำงานได้หากสัญญาณอินเทอร์เน็ตขาดหาย

ด้วยความก้าวหน้าทางเทคโนโลยีปัญญาประดิษฐ์ขนาดเล็กที่ฝังตัวบนอุปกรณ์ขอบ (Edge AI / TinyML) ทำให้ปัจจุบันสามารถนำโครงข่ายประสาทเทียม (Artificial Neural Networks) ที่ผ่านการฝึกสอนมาประมวลผลโดยตรงบนไมโครคอนโทรลเลอร์ขนาดเล็ก เช่น ESP32 ได้อย่างมีประสิทธิภาพ โดยไม่ต้องพึ่งพาเซิร์ฟเวอร์ภายนอก ด้วยเหตุนี้ ผู้จัดทำจึงได้นำเสนอ "อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์" โดยใช้โมดูลวัดความเฉื่อย 6 แกน (MPU-6050) ร่วมกับไมโครคอนโทรลเลอร์ ESP32 ทำการสกัดคุณลักษณะทางสถิติและพลศาสตร์ 18 มิติ (18 Features) แล้วประมวลผลผ่านโมเดลโครงข่ายประสาทเทียมแบบ Multi-Layer Perceptron (MLP) ที่ถูกแปลงเป็น Native C++ Forward Pass ซึ่งมีความเร็วในการประมวลผลต่ำกว่า 1 มิลลิวินาที พร้อมระบบตรวจจับซ้ำซ้อน (Dual Safety Fallback) และระบบติดตามตำแหน่งภายในอาคารด้วยค่าความแรงสัญญาณไร้สาย (RSSI) ร่วมกับระบบเว็บแอปพลิเคชันแดชบอร์ดและการแจ้งเตือนอัตโนมัติผ่าน Telegram และ Gmail เพื่อสร้างระบบเฝ้าระวังผู้สูงอายุที่มีความแม่นยำ ปลอดภัย ต้นทุนต่ำ และใช้งานได้จริงในชีวิตประจำวัน

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อออกแบบและสร้างอุปกรณ์ตรวจจับการล้มแบบสวมใส่ขนาดเล็กและต้นทุนต่ำ โดยใช้ไมโครคอนโทรลเลอร์ ESP32 ร่วมกับเซ็นเซอร์วัดความเฉื่อย 6 แกน (MPU-6050)

1.2.2 เพื่อวิจัย พัฒนา และฝึกสอนโมเดลปัญญาประดิษฐ์ขนาดเล็ก (TinyML) แบบ Multi-Layer Perceptron (MLP) ด้วยการสกัดคุณลักษณะ 18 มิติ และแปลงเป็น Native C++ Forward Pass สำหรับประมวลผลบนอุปกรณ์ฝังตัวแบบเรียลไทม์

1.2.3 เพื่อพัฒนาระบบติดตามตำแหน่งและวัดระยะทางสัมพัทธ์ภายในอาคารด้วยค่าความแรงสัญญาณไร้สาย (RSSI) ร่วมกับแบบจำลองการสูญเสียกำลังส่งในเส้นทาง (Log-Distance Path Loss Model) และการกรองด้วยคาลมาน (Kalman Filter) พร้อมระบบนับก้าว

1.2.4 เพื่อพัฒนาระบบเว็บแอปพลิเคชันแดชบอร์ด (Web Dashboard) จัดเก็บข้อมูลแบบผสมผสาน (Hybrid Database) แสดงผลสถานะแบบเรียลไทม์ผ่าน Socket.IO และส่งสัญญาณ

แจ้งเตือนเหตุฉุกเฉินผ่าน Telegram Bot, Gmail Alert และให้คำแนะนำสุขภาพด้วย Google Gemini AI

1.3 สมมติฐาน

1.3.1 การประมวลผลสัญญาณความเร่งและความเร็วเชิงมุมด้วยการสกัด 18 คุณลักษณะทางสถิติและพลังงาน ร่วมกับโครงข่ายประสาทเทียม MLP บนอุปกรณ์ ESP32 สามารถจำแนกเหตุการณ์การหกล้มออกจากกิจกรรมปกติในชีวิตประจำวัน (ADL) ได้อย่างถูกต้อง แม่นยำไม่น้อยกว่าร้อยละ 95

1.3.2 สถาปัตยกรรม Native C++ Forward Pass สามารถประมวลผลบนไมโครคอนโทรลเลอร์ ESP32 ได้โดยมีเวลาแฝง (Latency) น้อยกว่า 5 มิลลิวินาทีต่อหน้าต่างข้อมูล และใช้พื้นที่หน่วยความจำ Flash ไม่เกิน 10 กิโลไบต์

1.4 ข้อตกลงเบื้องต้น

1.4.1 ขอบเขตด้านฮาร์ดแวร์

1.4.1.1 ไมโครคอนโทรลเลอร์หลัก: ใช้บอร์ด ESP32 (Xtensa Dual-Core 32-bit LX6 @ 240 MHz, SRAM 520 KB)

1.4.1.2 เซ็นเซอร์วัดการเคลื่อนไหว: ใช้โมดูล MPU-6050 (3-axis Accelerometer ช่วงวัด ±16g และ 3-axis Gyroscope ช่วงวัด ±2000 deg/s) เชื่อมต่อผ่านบัส I2C ที่อัตราสุ่มตัวอย่าง 50 Hz (ทุก 20 มิลลิวินาที)

1.4.1.3 ระบบแจ้งเตือนในพื้นที่: ใช้ Active Buzzer สำหรับส่งเสียงสัญญาณเตือนภัยฉุกเฉิน และหลอดไฟ LED แสดงสถานะ

1.4.1.4 โหนดติดตามตำแหน่ง (Hardware2): ใช้บอร์ด ESP32 ทำหน้าที่สแกนค่าความแรงสัญญาณไร้สาย (RSSI Beacon) จากอุปกรณ์หลักเพื่อประมาณระยะทาง

1.4.2 ขอบเขตด้านโมเดลปัญญาประดิษฐ์และการประมวลผล (TinyML)

1.4.2.1 ขนาดหน้าต่างข้อมูล (Sliding Window): กำหนดขนาด 50 แซมเปิล (เทียบเท่าช่วงเวลา 1.0 วินาที ที่ความถี่ 50 Hz)

1.4.2.2 การสกัดคุณลักษณะ (Feature Extraction): สกัดเวกเตอร์คุณลักษณะ 18 มิติ ครอบคลุมค่าเฉลี่ย, ค่าสูงสุด, ค่าต่ำสุด, ส่วนเบี่ยงเบนมาตรฐาน, ค่าพิสัย, อัตราส่วนตกอิสระ, อัตราส่วนแรงกระแทก, SMA, ความเร่งแนวราบ, Jerk, Zero-Crossing Rate และสหสัมพันธ์ระหว่างแกน

1.4.2.3 โครงสร้างโมเดล: Multi-Layer Perceptron (MLP) ขนาด 18 -> 32 -> 16 -> 1 ใช้ฟังก์ชันกระตุ้น ReLU ในชั้นซ่อน และ Sigmoid ในชั้นผลลัพธ์

1.4.2.4 การฝังโมเดล (Deployment): แปลงค่าน้ำหนักและพารามิเตอร์ Standardization เป็น C Header (fall_model.h) ประมวลผลแบบ Native C++ บน ESP32 โดยไม่ต้องพึ่งพา TensorFlow Lite Runtime

1.4.3 ขอบเขตด้านชุดข้อมูล

1.4.3.1 ชุดข้อมูลมาตรฐานสากล SisFall Dataset: บันทึกจากอาสาสมัคร 38 คน (หนุ่มสาว 23 คน, ผู้สูงอายุ 15 คน) ครอบคลุม 19 กิจวัตรประจำวัน (D01-D19) และ 15 ท่าทางการหกล้ม (F01-F15) รวม 4,505 ไฟล์

1.4.3.2 ชุดข้อมูลการล้มจริง CollectData_Web: บันทึกจากเซ็นเซอร์ MPU-6050 จริงบนอาสาสมัคร 13 คน ใน 4 ทิศทางการล้ม (ล้มหน้า, ล้มหลัง, ล้มข้างซ้าย, ล้มข้างขวา) รวม 52 ไฟล์

1.4.4 ขอบเขตด้านการสื่อสารและเว็บแอปพลิเคชัน

1.4.4.1 การเชื่อมต่อเครือข่าย: ใช้ไลบรารี WiFiMulti สำหรับเชื่อมต่อโครงข่าย Wi-Fi หลายจุดอัตโนมัติ พร้อมระบบ Offline Storage บันทึกลง Flash Memory เมื่อขาดการเชื่อมต่อ

1.4.4.2 เว็บเซิร์ฟเวอร์และแดชบอร์ด: พัฒนาด้วย Node.js / Express และ PHP จัดเก็บข้อมูลด้วย MariaDB/MySQL และ MongoDB แสดงผลข้อมูลเรียลไทม์ด้วย Socket.IO

1.4.4.3 การแจ้งเตือนทางไกล: ส่งข้อความแจ้งเตือนพร้อมรายละเอียดความรุนแรงและทิศทางการล้มผ่าน Telegram Bot API และ Nodemailer (Gmail SMTP)

1.4.4.4 ปัญญาประดิษฐ์ผู้ช่วยสุขภาพ: ผสานรวม Google Gemini API ให้คำแนะนำด้านสุขภาพแก่ผู้ดูแล

1.5 ขอบเขตของโครงงาน

1.5.1 ได้อุปกรณ์ตรวจจับการล้มแบบสวมใส่ที่มีความแม่นยำสูง ต้นทุนต่ำ และสามารถประมวลผลการตัดสินใจได้บนตัวอุปกรณ์ทันที (Edge AI)

1.5.2 ลดอัตราการแจ้งเตือนผิดพลาด (False Alarm) จากกิจกรรมในชีวิตประจำวันทั่วไป ส่งผลให้ผู้ใช้งานเกิดความมั่นใจและยอมรับการใช้งานอุปกรณ์อย่างต่อเนื่อง

1.5.3 ช่วยลดระยะเวลาในการเข้าช่วยเหลือผู้สูงอายุเมื่อเกิดเหตุหกล้ม ป้องกันการเกิดภาวะนอนค้างที่พื้นเป็นเวลานาน (Long Lie Syndrome) และลดอัตราความพิการหรือเสียชีวิต

1.5.4 ได้ระบบแดชบอร์ดติดตามตำแหน่งและสุขภาพที่ช่วยให้ผู้ดูแลสามารถติดตามสถานะของผู้สูงอายุได้แบบเรียลไทม์ผ่านสมาร์ตโฟนหรือคอมพิวเตอร์

1.5.5 ได้แนวทางการพัฒนาและองค์ความรู้ด้านการประยุกต์ใช้ TinyML และ Native C++ บนไมโครคอนโทรลเลอร์ราคาประหยัดสำหรับงานด้านวิศวกรรมการแพทย์และ IoT

1.6 ตัวแปรที่ศึกษา

ในการวิจัยและพัฒนาอุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ ได้กำหนดตัวแปรในการศึกษาออกเป็น 3 ด้านหลัก ดังนี้:

1.6.1 ตัวแปรในการประเมินประสิทธิภาพโมเดลปัญญาประดิษฐ์ (TinyML Model)

ตัวแปรต้น (Independent Variables):

1. รูปแบบคุณลักษณะทางสถิติและพลศาสตร์ของสัญญาณการเคลื่อนไหว 18 มิติ (18 Biomechanical Features) ที่สกัดจากเซ็นเซอร์ MPU-6050
2. โครงสร้างสถาปัตยกรรมโครงข่ายประสาทเทียม Multi-Layer Perceptron (MLP) ขนาด 18 ⟶32 ⟶ 16 ⟶ 1
3. ชุดข้อมูลสัญญาณการเคลื่อนไหวลูกผสม (Hybrid Dataset: SisFall Dataset ร่วมกับ CollectData_Web)

ตัวแปรตาม (Dependent Variables):

1. ค่าความถูกต้องในการจำแนกเหตุการณ์ (Accuracy)
2. ค่าความไวหรือความสามารถในการตรวจจับการล้ม (Sensitivity / Recall)
3. ค่าความจำเพาะในการแยกแยะกิจกรรมประจำวัน (Specificity)
4. ค่าความแม่นยำ (Precision) และค่าคะแนนเฉลี่ย F1-Score

ตัวแปรควบคุม (Controlled Variables):

1. ความถี่ในการสุ่มตัวอย่างสัญญาณเซ็นเซอร์ (Sampling Rate 50 Hz)
2. ขนาดของหน้าต่างข้อมูลแบบเลื่อน (Sliding Window Size 128 แซมเปิล หรือ 2.56 วินาที โดยมีการทับซ้อน 50%)
3. สภาวะแวดล้อมและพารามิเตอร์การฝึกสอนโมเดล (Optimizer, Learning Rate 0.001, Binary Cross-Entropy Loss)

1.6.2 ตัวแปรในการประมวลผลบนไมโครคอนโทรลเลอร์ (Edge Computing & Hardware 1)

ตัวแปรต้น (Independent Variables):

1. สถาปัตยกรรมการคำนวณแบบ Native C++ Forward Pass เปรียบเทียบกับเฟรมเวิร์กมาตรฐาน (TensorFlow Lite for Microcontrollers)
2. รูปแบบกิจกรรมการเคลื่อนไหวจริง (5 ท่าทางการหกล้ม และ 5 รูปแบบกิจกรรมในชีวิตประจำวัน: ADLs)

ตัวแปรตาม (Dependent Variables):

1. ระยะเวลาหน่วงในการประมวลผลและอนุมานผลลัพธ์ (Inference Latency ในหน่วยมิลลิวินาที)
2. ปริมาณการใช้พื้นที่หน่วยความจำ Flash (ROM) และหน่วยความจำแรม (SRAM) ของ ESP32
3. อัตราความสำเร็จในการตรวจจับการล้มจริงบนอุปกรณ์ฮาร์ดแวร์ (Detection Rate)

4. อัตราการแจ้งเตือนผิดพลาดจากกิจกรรมปกติ (False Alarm / False Positive Rate)

ตัวแปรควบคุม (Controlled Variables):

1. ตำแหน่งการสวมใส่อุปกรณ์ (บริเวณบั้นเอว / Center of Mass)
2. ชิปไมโครคอนโทรลเลอร์ ESP32 ความเร็วสัญญาณนาฬิกา 240 MHz
3. ย่านวัดของเซ็นเซอร์ MPU-6050 (ความเร่ง$\pm 16g$,อัตราเร็วเชิงมุม $\pm \frac{2000°}{s}$)

1.6.3 ตัวแปรในการระบุตำแหน่งภายในอาคารและการสื่อสาร (Localization & Cloud System)

ตัวแปรต้น (Independent Variables):

1. ระดับความแรงสัญญาณไร้สาย (RSSI) ที่วัดได้ในแต่ละระยะห่าง (1 – 5 เมตร)
2. การประยุกต์ใช้ขั้นตอนวิธีตัวกรองคาลมาน (Kalman Filter) ในการปรับเรียบสัญญาณ
3. ช่องทางการส่งข้อมูลแจ้งเตือน (Web Dashboard ผ่าน Socket.IO, Telegram Bot API, Gmail SMTP)

ตัวแปรตาม (Dependent Variables):

1. ความถูกต้องในการระบุโซนห้องภายในอาคาร (Room Localization Accuracy)
2. ระยะเวลาการส่งผ่านข้อมูลและการแจ้งเตือนตั้งแต่เกิดเหตุจนถึงผู้รับ (End-to-End Latency)

ตัวแปรควบคุม (Controlled Variables):

1. สภาพแวดล้อมอาคารทดสอบและพารามิเตอร์แบบจำลอง Path Loss ($n = 2.4, \mathrm{RSSI}_0 = -59\ \mathrm{dBm}$)
2. ความเร็วและแบนด์วิดท์เครือข่ายสัญญาณไร้สาย Wi-Fi 2.4 GHz

1.7 ประโยชน์ที่คาดว่าจะได้รับ

1.7.1 ด้านสุขภาพและความปลอดภัยของผู้สูงอายุ:

1.7.1.1 ช่วยลดความเสี่ยงจากการเกิดภาวะนอนค้างที่พื้นเป็นเวลานาน (Long Lie Syndrome) ซึ่งเป็นสาเหตุสำคัญของการเสียชีวิตและภาวะแทรกซ้อนรุนแรง

1.7.1.2 เพิ่มโอกาสรอดชีวิตและลดอัตราความพิการถาวร โดยระบบสามารถส่งสัญญาณแจ้งเตือนและระบุตำแหน่งห้องได้อย่างรวดเร็วภายใน 1.25 วินาที ทำให้ผู้ดูแลหรือหน่วยกู้ชีพเข้าช่วยเหลือได้ทันท่วงที

1.7.2 ด้านประสิทธิภาพและความน่าเชื่อถือของเทคโนโลยี:

1.7.2.1 ได้ระบบตรวจจับการล้มที่มีความแม่นยำสูง (Accuracy 98.98% และ Sensitivity 100%) พร้อมลดอัตราการแจ้งเตือนผิดพลาด (False Alarm) จากกิจกรรมประจำวัน ทำให้ผู้สูงอายุเกิดความมั่นใจและยอมรับการใช้งานอย่างต่อเนื่อง

1.7.2.2 ได้สถาปัตยกรรม Native C++ Forward Pass ที่ใช้หน่วยความจำต่ำมาก (Flash เพียง 4.6 KB) และประมวลผลได้รวดเร็วระดับ On-Device (< 1 ms) ทำให้ระบบทำงานได้แบบเรียลไทม์และทำงานได้ต่อเนื่องแม้อยู่ในจุดอับสัญญาณอินเทอร์เน็ต (Offline Mode)

1.7.3 ด้านเศรษฐกิจและสังคม:

1.7.3.1 ได้ต้นแบบอุปกรณ์สวมใส่ตรวจจับการล้มที่มีต้นทุนการผลิตต่ำ (ประมาณ 400–500 บาทต่อชุด) เมื่อเทียบกับสมาร์ตวอทช์หรืออุปกรณ์ทางการค้าที่มีราคาสูง ทำให้ประชาชนทุกระดับและชุมชนสามารถเข้าถึงได้ง่าย

1.7.3.2 ช่วยลดภาระและความวิตกกังวลของผู้ดูแลและครอบครัว (Caregiver Burden) โดยสามารถตรวจสอบสถานะและตำแหน่งของผู้สูงอายุได้ตลอด 24 ชั่วโมงผ่านเว็บแดชบอร์ด

1.7.4 ด้านวิชาการและการวิจัยต่อยอด:

1.7.4.1 ได้องค์ความรู้และระเบียบวิธีวิจัยด้านการประยุกต์ใช้ปัญญาประดิษฐ์ฝังตัว (TinyML) ร่วมกับการสกัดคุณลักษณะเชิงกลศาสตร์ชีวภาพ 18 มิติ บนชิปไมโครคอนโทรลเลอร์ขนาดเล็ก

1.7.4.2 เป็นแนวทางต้นแบบสำหรับการพัฒนาอุปกรณ์ IoT ทางการแพทย์และอุปกรณ์สวมใส่เพื่อสุขภาพ (Wearable Healthcare Devices) ในยุคสังคมผู้สูงอายุต่อไป

1.8 คำนิยามศัพท์

1.8.1 การหกล้ม (Fall): เหตุการณ์ที่ร่างกายของผู้สูงอายุเสียสมดุลและตกลงสู่พื้นหรือระดับที่ต่ำกว่าโดยไม่ได้ตั้งใจ ก่อให้เกิดแรงกระแทกอย่างฉับพลัน

1.8.2 กิจกรรมในชีวิตประจำวัน (Activities of Daily Living: ADL): กิจกรรมการเคลื่อนไหวตามปกติของมนุษย์ เช่น การเดิน การนั่ง การยืน การนอน การขึ้นลงบันได และการก้มหยิบของ

1.8.3 หน่วยวัดความเฉื่อย (Inertial Measurement Unit: IMU): เซ็นเซอร์ตรวจจับการเคลื่อนไหวทางกายภาพ ประกอบด้วยเซ็นเซอร์วัดความเร่ง (Accelerometer) 3 แกน และเซ็นเซอร์วัดความเร็วเชิงมุม (Gyroscope) 3 แกน

1.8.4 ปัญญาประดิษฐ์ขนาดเล็ก (TinyML): เทคโนโลยีการนำโมเดลการเรียนรู้ของเครื่อง (Machine Learning) ขนาดกะทัดรัดไปประมวลผลบนไมโครคอนโทรลเลอร์ที่มีทรัพยากรจำกัด

1.8.5 การสกัดคุณลักษณะ (Feature Extraction): กระบวนการคำนวณและแปลงสัญญาณดิบเชิงอนุกรมเวลาให้อยู่ในรูปของตัวแปรทางสถิติและพลศาสตร์เพื่อใช้เป็นข้อมูลนำเข้าของโมเดลปัญญาประดิษฐ์

1.8.6 การส่งต่อข้อมูลไปข้างหน้าแบบเนทีฟ (Native Forward Pass): การคำนวณโครงข่ายประสาทเทียมด้วยคำสั่งภาษา C/C++ พื้นฐานโดยตรงบนไมโครคอนโทรลเลอร์ โดยไม่พึ่งพาเฟรมเวิร์กขนาดใหญ่

1.8.7 ดัชนีความแรงสัญญาณที่ได้รับ (RSSI): ค่าตัวเลขที่ระบุระดับความเข้มของสัญญาณคลื่นวิทยุไร้สาย ใช้สำหรับประมาณระยะทางสัมพัทธ์ระหว่างตัวส่งและตัวรับ

1.8.8 ภาวะนอนค้างที่พื้นเป็นเวลานาน (Long Lie Syndrome): สภาวะที่ผู้ป่วยหรือผู้สูงอายุหกล้มและไม่สามารถลุกขึ้นได้จนต้องนอนอยู่บนพื้นเป็นเวลานานเกินกว่า 1 ชั่วโมง ก่อให้เกิดภาวะแทรกซ้อนรุนแรง

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การวิจัยและพัฒนาอุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ จำเป็นต้องอาศัยหลักการทางทฤษฎีหลากหลายสาขาวิชา ได้แก่ พลศาสตร์ของการเคลื่อนไหวและสรีรวิทยาของการล้มในมนุษย์, ทฤษฎีเซ็นเซอร์วัดความเฉื่อย (IMU), สถาปัตยกรรมระบบสมองกลฝังตัวและไมโครคอนโทรลเลอร์ ESP32, การประมวลผลปัญญาประดิษฐ์บนอุปกรณ์ขอบ (Edge AI / TinyML), ทฤษฎีการสื่อสารไร้สายและการระบุตำแหน่งภายในอาคาร (Indoor Localization) ตลอดจนการทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้อง โดยมีรายละเอียดดังต่อไปนี้

2.1 สรีรวิทยาและพลศาสตร์ของการล้มในผู้สูงอายุ

ในทางชีวกลศาสตร์ (Biomechanics) เหตุการณ์การหกล้มของมนุษย์สามารถจำแนกออกเป็น 4 ระยะตามลำดับเวลา (Fall Phases) ได้แก่:

1. ระยะก่อนการล้ม (Pre-fall Phase): เป็นช่วงที่ร่างกายเริ่มสูญเสียการทรงตัว (Loss of Balance) จากปัจจัยต่าง ๆ เช่น การสะดุดสิ่งกีดขวาง (Trip) หรือการลื่นไถล (Slip) ซึ่งความเร่งของร่างกายยังคงมีค่าใกล้เคียงกับแรงโน้มถ่วงของโลก (1.0g)

2. ระยะตกอิสระ (Free-fall Phase): ร่างกายเริ่มเคลื่อนที่ลงสู่พื้นอย่างรวดเร็วภายใต้แรงโน้มถ่วง ทำให้ค่าความเร่งลัพธ์รวม (Signal Vector Magnitude: SVM) ลดลงอย่างฉับพลันเข้าใกล้สภาวะไร้น้ำหนัก (SVM < 0.5g หรือต่ำกว่า 0.3g) โดยช่วงเวลานี้มักมีความยาวประมาณ 100 ถึง 300 มิลลิวินาที

3. ระยะกระแทกพื้น (Impact Phase): ส่วนใดส่วนหนึ่งของร่างกายกระทบกับพื้นผิวหรือวัตถุ เกิดการเปลี่ยนโมเมนตัมอย่างรวดเร็ว ส่งผลให้ค่าขนาดความเร่งพุ่งขึ้นสูงมากอย่างฉับพลัน (Impact Peak SVM > 2.5g ถึง 3.5g ขึ้นไป) พร้อมทั้งความเร็วเชิงมุมของไจโรสโคปหมุนอย่างรุนแรง

4. ระยะหลังการล้ม (Post-impact / Rest Phase): ร่างกายหยุดการเคลื่อนไหวและนอนนิ่งอยู่บนพื้น หากผู้สูงอายุไม่สามารถลุกขึ้นได้ ค่าความเร่งจะคงที่อยู่ที่ประมาณ 1.0g ในทิศทางของแกนที่นอนราบขนานกับพื้นโลก

นอกจากนี้ การล้มในผู้สูงอายุยังสามารถจำแนกทิศทางตามลักษณะทางกายภาพออกเป็น 5 รูปแบบหลัก ได้แก่ การล้มไปข้างหน้า (Forward Fall), การล้มไปข้างหลัง (Backward Fall), การล้มไปทางด้านข้างซ้าย/ขวา (Lateral Fall), และการทรุดตัวลงในแนวดิ่ง (Vertical Fall) การจำแนกทิศทางเหล่านี้มีความสำคัญอย่างยิ่งต่อการประเมินตำแหน่งที่อาจได้รับบาดเจ็บรุนแรง เช่น การล้มไปข้างหลังมีความเสี่ยงสูงต่อกระดูกกะโหลกศีรษะและกระดูกสันหลัง ขณะที่การล้มไปด้านข้างมีความเสี่ยงสูงต่อกระดูกข้อสะโพกหัก

2.2 เซ็นเซอร์วัดการเคลื่อนไหว (Inertial Measurement Unit: IMU)

หน่วยวัดความเฉื่อย (Inertial Measurement Unit: IMU) เป็นอุปกรณ์ตรวจวัดการเคลื่อนไหวทางกายภาพโดยใช้เทคโนโลยีระบบเครื่องกลไฟฟ้าจุลภาค (Micro-Electro-Mechanical Systems: MEMS) ในโครงงานนี้เลือกใช้โมดูล MPU-6050 ซึ่งรวมเซ็นเซอร์ 2 ชนิดไว้ในชิปตัวเดียว ได้แก่:

1. มาตรวัดความเร่ง 3 แกน (3-Axis Accelerometer): วัดความเร่งเชิงเส้น (Linear Acceleration) ในแกน X, Y, Z จากการเปลี่ยนแปลงค่าความจุไฟฟ้า (Capacitive Sensing) เมื่อมวลทดสอบเคลื่อนที่ภายใต้แรงโน้มถ่วงและความเร่งภายนอก รองรับช่วงการวัด (Full-Scale Range) ±2g, ±4g, ±8g และ ±16g

2. ไจโรสโคป 3 แกน (3-Axis Gyroscope): วัดอัตราความเร็วเชิงมุม (Angular Velocity) ในการหมุนรอบแกน X, Y, Z โดยอาศัยหลักการแรงโคริโอลิส (Coriolis Effect) รองรับช่วงการวัด ±250, ±500, ±1000 และ ±2000 องศาต่อวินาที (°/s)

การคำนวณขนาดเวกเตอร์ความเร่งลัพธ์ (Signal Vector Magnitude: SVM หรือ Accel Magnitude) ในหน่วยเท่าของแรงโน้มถ่วง (g) คำนวณได้จากสูตรคณิตศาสตร์:

$$SVM = \frac{sqrt(Ax^2 + Ay^2 + Az^2)}{9.81}$$

โดย Ax, Ay, Az คือค่าความเร่งในแต่ละแกน (m/s^2) ซึ่งในสภาวะอยู่นิ่งค่า SVM จะมีค่าเท่ากับ 1.0g เสมอ

2.3 ไมโครคอนโทรลเลอร์และการประมวลผลฝังตัว ESP32

ไมโครคอนโทรลเลอร์ ESP32 (ผลิตโดย Espressif Systems) เป็นชิปประมวลผลสมรรถนะสูงที่ออกแบบมาสำหรับงาน Internet of Things (IoT) และ Edge AI โดยมีคุณลักษณะสำคัญทางเทคนิคดังนี้:

1. หน่วยประมวลผล: สถาปัตยกรรม 32-bit Xtensa Dual-Core LX6 ทำงานที่ความถี่สัญญาณนาฬิกาสูงสุด 240 MHz มีสมรรถนะการคำนวณสูงถึง 600 DMIPS รองรับคำสั่งคำนวณทางคณิตศาสตร์แบบทศนิยม (Hardware Floating Point Unit: FPU)

2. หน่วยความจำ: มีหน่วยความจำ SRAM ภายในขนาด 520 KB และ Flash Memory ภายนอกขนาด 4 MB ถึง 16 MB สำหรับจัดเก็บโปรแกรมและโมเดลปัญญาประดิษฐ์

3. การสื่อสารไร้สาย: มีโมดูล Wi-Fi มาตรฐาน IEEE 802.11 b/g/n (2.4 GHz) และ Bluetooth v4.2 BR/EDR / BLE ในตัว สามารถทำงานได้ทั้งในโหมดสถานี (Station: STA) และโหมดจุดกระจายสัญญาณ (Access Point: AP)

4. การจัดการพลังงาน: รองรับโหมดประหยัดพลังงานหลากหลายระดับ (Light Sleep, Deep Sleep, Hibernation) ทำให้เหมาะสำหรับการใช้งานในอุปกรณ์สวมใส่พกพา

2.4 ปัญญาประดิษฐ์ขนาดเล็กและการประมวลผลบนอุปกรณ์ขอบ (TinyML / Edge AI)

TinyML (Tiny Machine Learning) เป็นการนำโมเดลโครงข่ายประสาทเทียมและขั้นตอนวิธี Machine Learning มาประมวลผลบนไมโครคอนโทรลเลอร์ที่มีหน่วยความจำระดับกิโลไบต์และการใช้พลังงานต่ำระดับมิลลิวัตต์ โดยมีข้อได้เปรียบสำคัญ 4 ประการ คือ: (1) มีเวลาแฝงต่ำมาก (Ultra-low Latency) ตัดสินใจได้ทันที, (2) ปลอดภัยต่อความเป็นส่วนตัว (Enhanced Privacy) ข้อมูลดิบไม่ถูกส่งออกนอกเครื่อง, (3) ทนทานต่อการตัดการเชื่อมต่อ (Offline Reliability) ทำงานได้แม้ไม่มีสัญญาณอินเทอร์เน็ต, และ (4) ประหยัดพลังงานและแบนด์วิดท์ (Bandwidth & Energy Efficiency)

2.4.1 สถาปัตยกรรมโครงข่ายประสาทเทียม Multi-Layer Perceptron (MLP)

โครงข่ายประสาทเทียมแบบ Multi-Layer Perceptron (MLP) เป็นโครงข่ายประสาทเทียมแบบป้อนตรง (Feedforward Neural Network) ที่ประกอบด้วยชั้นนำเข้า (Input Layer), ชั้นซ่อน (Hidden Layers), และชั้นผลลัพธ์ (Output Layer) ในโครงงานนี้โมเดลถูกกำหนดขนาดเป็น 18 → 32 → 16 → 1 โดยแต่ละชั้นมีการคำนวณแบบ Linear Combination ร่วมกับฟังก์ชันกระตุ้น (Activation Function):

1. ฟังก์ชัน Rectified Linear Unit (ReLU): ใช้ในชั้นซ่อนที่ 1 และ 2 เพื่อเพิ่มคุณสมบัติไม่เป็นเชิงเส้น (Non-linearity) และช่วยให้คำนวณได้รวดเร็ว: ReLU(x) = max(0, x)

2. ฟังก์ชัน Sigmoid: ใช้ในชั้นผลลัพธ์เพื่อแปลงค่าให้อยู่ในช่วงความน่าจะเป็น 0.0 ถึง 1.0 สำหรับการจำแนกการล้ม: Sigmoid(x) = 1 / (1 + exp(-x))

2.4.2 การสกัด 18 คุณลักษณะทางสถิติและพลศาสตร์ (Feature Engineering)

ในการตรวจจับการล้ม สัญญาณความเร่งและไจโรสโคปจากหน้าต่างข้อมูล 50 แซมเปิล (1.0 วินาที ที่ 50 Hz) จะถูกสกัดเป็นเวกเตอร์คุณลักษณะ 18 มิติ ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1 รายละเอียดการสกัด 18 คุณลักษณะทางสถิติและพลศาสตร์

ลำดับ	ชื่อคุณลักษณะ (Feature Name)	สูตรทางคณิตศาสตร์ / ความหมาย	บทบาทในการแยกแยะการล้ม
0	MeanAccel Mag	Mean(SVM)	วัดระดับความเร่งเฉลี่ยของการเคลื่อนไหวรวม
1	Max Accel Mag	Max(SVM)	ตรวจจับแรงกระแทกพื้นสูงสุด (Impact Peak > 3.5g)
2	Min Accel Mag	Min(SVM)	ตรวจจับสภาวะการตกอิสระไร้น้ำหนัก (Free-fall < 0.3g)
3	Std Accel Mag	StdDev(SVM)	วัดความแปรปรวนและการผันผวนของความเร่ง

4	Range Accel Mag	Max(SVM) - Min(SVM)	วัดพิสัยความแตกต่างของแรงกระแทกฉับพลัน
5	Free-fall Ratio	Count(SVM < 0.3g) / 50	สัดส่วนเวลาที่อยู่ในสภาวะตกอิสระ
6	Impact Ratio	Count(SVM > 3.5g) / 50	สัดส่วนเวลาที่มีแรงกระแทกสูงเกินเกณฑ์
7	Mean Gyro Mag	Mean(sqrt(Gx^2+Gy^2+Gz^2))	ความเร็วเชิงมุมเฉลี่ยในการหมุนตัว
8	Max Gyro Mag	Max(sqrt(Gx^2+Gy^2+Gz^2))	ความเร็วเชิงมุมสูงสุดขณะสะบัดหรือพลิกตัว
9	Std Gyro Mag	StdDev(Gyro Magnitude)	ความผันผวนของความเร็วเชิงมุม
10	Signal Magnitude Area (SMA)	Sum(|Ax|+|Ay|+|Az|) / (50*9.81)	พลังงานสะสมรวมของการเคลื่อนไหว
11	Mean Horizontal Accel	Mean(sqrt(Ax^2+Az^2) / 9.81)	ความเร่งเฉลี่ยในระนาบแนวนอน
12	Max Horizontal Accel	Max(sqrt(Ax^2+Az^2) / 9.81)	แรงกระแทกในระนาบแนวนอน (ล้มหน้า/หลัง/ข้าง)
13	Mean Jerk	Mean(|SVM[i] - SVM[i-1]|)	อัตราการเปลี่ยนความเร่งเฉลี่ย (ความกระตุก)
14	Max Jerk	Max(|SVM[i] - SVM[i-1]|)	อัตราการเปลี่ยนความเร่งสูงสุดตอนกระแทก
15	Zero-Crossing Rate	Count(SignChange(SVM - 1.0g))	ความถี่ในการแกว่งตัวข้ามระดับแรงโน้มถ่วง
16	Correlation (Ax, Ay)	Cov(Ax, Ay) / (Std(Ax)*Std(Ay))	ความสัมพันธ์เชิงทิศทางระหว่างแกน X และ Y
17	Correlation (Ay, Az)	Cov(Ay, Az) / (Std(Ay)*Std(Az))	ความสัมพันธ์เชิงทิศทางระหว่างแกน Y และ Z

2.5 การระบุตำแหน่งภายในอาคารและการนับก้าว (Indoor Localization & Step Counting)

การติดตามตำแหน่งภายในอาคารของผู้สูงอายุโดยไม่พึ่งพาระบบดาวเทียม GPS ที่มีข้อจำกัดเรื่องสิ่งกีดขวาง อาศัยการวัดค่าความแรงสัญญาณไร้สาย (Received Signal Strength Indication: RSSI) ที่ส่งมาจากอุปกรณ์หลัก (Hardware1 Beacon) ไปยังโหนดรับสัญญาณ (Hardware2 Node)

2.5.1 แบบจำลองการสูญเสียกำลังส่งในเส้นทาง (Log-Distance Path Loss Model) ความสัมพันธ์ระหว่างค่าความแรงสัญญาณ RSSI (dBm) และระยะทางสัมพัทธ์ d (เมตร) สามารถประมาณได้จากสมการการสูญเสียกำลังส่ง:

RSSI = -10 * n * log10(d) + A

โดย A คือค่าความแรงสัญญาณที่ระยะอ้างอิง 1 เมตร และ n คือค่าสัมประสิทธิ์การสูญเสียสัญญาณตามสภาพแวดล้อม (Path Loss Exponent: ปกติมีค่าระหว่าง 2.0 ถึง 3.5 ในอาคาร) เมื่อแปลงกลับจะได้สมการคำนวณระยะทาง d:

d = 10 ^ ((A - RSSI) / (10 * n))

2.5.2 การกรองสัญญาณรบกวนด้วยคาลมาน (Kalman Filter)

เนื่องจากสัญญาณ RSSI ภายในอาคารมีความผันผวนสูงจากการสะท้อนของคลื่น (Multipath Fading) ระบบจึงประยุกต์ใช้คาลมานฟิลเตอร์ 1 มิติ เพื่อกรองสัญญาณรบกวน โดยมีขั้นตอนการคำนวณ 2 ส่วนหลัก ได้แก่:

1. ขั้นตอนการทำนายสถานะ (Prediction): ทำนายค่าระยะทางล่วงหน้าและความแปรปรวนของความผิดพลาด (P = P + Q โดย Q คือ Process Noise Covariance)

2. ขั้นตอนการปรับปรุงค่าตามผลการวัด (Measurement Update): คำนวณค่าคาลมานเกน (Kalman Gain: K = P / (P + R) โดย R คือ Measurement Noise Covariance) แล้วปรับปรุงค่าประมาณสถานะ x = x + K * (z - x) และปรับปรุงความแปรปรวน P = (1 - K) * P

2.6 ระบบเครือข่ายและสถาปัตยกรรมเว็บแอปพลิเคชัน

ระบบซอฟต์แวร์ส่วนหลังบ้าน (Backend Server) และแดชบอร์ดแสดงผลถูกออกแบบตามสถาปัตยกรรมไมโครเซอร์วิสและเว็บมาตรฐาน ประกอบด้วย:

2.6.1 Node.js & Express: ให้บริการ RESTful API สำหรับการรับส่งข้อมูลระหว่างฮาร์ดแวร์และแดชบอร์ด มีประสิทธิภาพสูงในการประมวลผลแบบ Asynchronous I/O

2.6.2 Socket.IO: เทคโนโลยี WebSocket สองทิศทาง ช่วยให้ข้อมูลการก้าวเดิน พิกัดตำแหน่ง และการแจ้งเตือนการล้มแสดงผลบนแดชบอร์ดได้แบบเรียลไทม์ (Real-time Push Notification)

2.6.3 สถาปัตยกรรมฐานข้อมูลแบบผสมผสาน (Hybrid Database): จัดเก็บข้อมูลเชิงโครงสร้างและประวัติเหตุการณ์ (Event Logs) ใน MariaDB/MySQL และจัดเก็บข้อมูลสตรีมดิบจากเซ็นเซอร์และประวัติการแชทใน MongoDB

2.6.4 การแจ้งเตือนฉุกเฉินทางไกล: เชื่อมต่อ Telegram Bot API ส่งข้อความด่วนพร้อมข้อมูลความมั่นใจ AI และระบบ Nodemailer (Gmail SMTP) ส่งอีเมลแจ้งเตือนผู้ดูแล

2.6.5 การบูรณาการปัญญาประดิษฐ์ Google Gemini API: ให้คำแนะนำด้านสุขภาพเบื้องต้น การวิเคราะห์พฤติกรรมการก้าวเดิน และคำแนะนำฉุกเฉินแก่ผู้ดูแล

2.7 งานวิจัยที่เกี่ยวข้อง (Literature Review)

จากการทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้องกับการตรวจจับการล้มและการเฝ้าระวังผู้สูงอายุทั้งในและต่างประเทศ สามารถสรุปเปรียบเทียบจุดเด่นและข้อจำกัดของแต่ละงานวิจัยได้ดังแสดงในตารางที่ 2.2

ตารางที่ 2.2 การเปรียบเทียบงานวิจัยที่เกี่ยวข้องกับระบบตรวจจับการล้ม

ผู้วิจัยและปี	เทคโนโลยี / เซ็นเซอร์	อัลกอริทึมที่ใช้	ความแม่นยำ	ข้อดีและข้อจำกัด
Sucerquia et al. (2017)	ADXL345 + ITG3200 (เอว)	เกณฑ์คงที่ (Threshold-based)	Accuracy ~84%	สร้างชุดข้อมูล SisFall มาตรฐาน แต่ยังมี False Alarm สูง
Ordóñez et al. (2016)	IMU สวมใส่หลายจุด	Deep ConvLSTM Neural Net	Accuracy 95.8%	แม่นยำสูง แต่กินทรัพยากรมาก ไม่สามารถรันบนไมโครคอนโทรลเลอร์ได้
Kangas et al. (2008)	Tri-axial Accelerometer	Threshold + Posture Detection	Sensitivity 91%	ประมวลผลง่าย แต่แยกการนั่งเร็วกับการล้มได้ไม่ดี
สมชาย และคณะ (2566)	ESP8266 + MPU-6050	ส่งข้อมูลดิบขึ้น Cloud Server	Accuracy 88.5%	ต้นทุนต่ำ แต่มี Latency สูง และไม่ทำงานเมื่อเน็ตหลุด
งานวิจัยนี้ (2569)	ESP32 + MPU-6050 + RSSI	TinyML MLP (18 Features) C++	Accuracy 98.98%	ประมวลผลบนเครื่อง (Latency <1ms), Hybrid Dataset, แจ้งเตือนหลายช่องทาง

บทที่ 3
วิธีดำเนินงาน

การดำเนินงานวิจัยเรื่อง อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ (Elderly Fall Detection Device with Embedded AI on Microcontroller) มีขั้นตอนการออกแบบและพัฒนาอย่างเป็นระบบ โดยแบ่งเนื้อหาการดำเนินงานออกเป็น 8 ส่วนหลัก ได้แก่ สถาปัตยกรรมระบบโดยรวม, การออกแบบและพัฒนาฮาร์ดแวร์ตรวจจับการล้ม (Hardware 1), การออกแบบและพัฒนาฮาร์ดแวร์ระบุตำแหน่ง (Hardware 2), การเตรียมชุดข้อมูลและการประมวลผลสัญญาณ, การสร้างและฝึกสอนโมเดลปัญญาประดิษฐ์ TinyML, การพัฒนาเฟิร์มแวร์ C++ Forward Pass บน ESP32, การพัฒนาระบบคลาวด์และเว็บแอปพลิเคชัน และขั้นตอนการทดสอบและประเมินผลประสิทธิภาพระบบ

3.1 สถาปัตยกรรมระบบโดยรวม (System Architecture)

3.1.1 ะบบตรวจจับการล้มและแจ้งเตือนฉุกเฉินที่พัฒนาขึ้นได้รับการออกแบบภายใต้สถาปัตยกรรม 3 ระดับ (Three-Tier Architecture) เพื่อให้ระบบสามารถประมวลผลและตัดสินใจได้อย่างรวดเร็วในระดับ Edge Device พร้อมทั้งสามารถสื่อสารและแจ้งเตือนไปยังผู้ดูแลผ่านระบบคลาวด์แบบเรียลไทม์ ดังนี้:

1. ระดับอุปกรณ์ฝังตัว (Edge Device Layer): ประกอบด้วยอุปกรณ์สวมใส่ตรวจจับการล้ม (Hardware 1) ซึ่งใช้ไมโครคอนโทรลเลอร์ ESP32 เชื่อมต่อกับเซนเซอร์วัดความเฉื่อย 6 แกน (MPU-6050) ทำหน้าที่ประมวลผลสัญญาณความเร่งและอัตราเร็วเชิงมุม สกัดคุณลักษณะทางกายภาพ 18 มิติ และประมวลผลโมเดลโครงข่ายประสาทเทียม TinyML MLP แบบออฟไลน์ภายในชิป (On-Device Inference) พร้อมระบบสำรองข้อมูลใน Flash Memory กรณีสัญญาณเครือข่ายขัดข้อง และอุปกรณ์ระบุตำแหน่งภายในอาคาร (Hardware 2) ที่ส่งสัญญาณ RSSI ผ่านเครือข่ายไร้สาย
2. ระดับระบบคลาวด์และบริการแบ็กเอนด์ (Cloud & Backend Services Layer): พัฒนาขึ้นด้วย Node.js และ Express.js ทำหน้าที่เป็นตัวกลางในการรับ-ส่งข้อมูลผ่าน RESTful API และ WebSockets (Socket.IO) ทำงานร่วมกับฐานข้อมูลคู่ (Dual Database) ได้แก่ MariaDB สำหรับจัดเก็บข้อมูลเชิงโครงสร้าง (ประวัติผู้สูงอายุ ข้อมูลอุปกรณ์ ผู้ดูแล) และ MongoDB สำหรับจัดเก็บข้อมูลอนุกรมเวลา (Time-Series Sensor Logs) พร้อมโมดูลแจ้งเตือนหลายช่องทาง (Telegram Bot API, Gmail SMTP Nodemailer) และการวิเคราะห์สรุปเหตุการณ์ด้วย Google Gemini AI

3. ระดับการแสดงผลและส่วนประสานผู้ใช้ (User Interface Layer): เว็บแอปพลิเคชันแดชบอร์ดที่ออกแบบในรูปแบบ Dark Cyberpunk Theme นำเสนอข้อมูลสถานะผู้สูงอายุแบบเรียลไทม์ แผนผังระบุพิกัดและตำแหน่งห้อง (Indoor Localization & OpenStreetMap) กราฟแสดงผลค่าเซนเซอร์สด แผงควบคุมและปุ่มยกเลิกการแจ้งเตือนฉุกเฉิน (False Alarm Cancel) และระบบออกรายงานสรุปสถิติเหตุการณ์ข้อความ

3.2 การออกแบบและพัฒนาฮาร์ดแวร์ตรวจจับการล้ม

ฮาร์ดแวร์ส่วนที่ 1 ได้รับการออกแบบให้มีขนาดกะทัดรัด น้ำหนักเบา และสามารถติดตั้งบนสายรัดเอวหรือกระเป๋าเสื้อของผู้สูงอายุได้อย่างสะดวก เพื่อให้ได้ตำแหน่งกึ่งกลางมวลของร่างกาย (Center of Mass) ซึ่งเหมาะสมที่สุดสำหรับการตรวจจับการเคลื่อนไหวและการล้ม โดยมีรายละเอียดการต่อวงจรและกำหนดขาไอโอ (Pin Assignment) ดังตารางที่ 3.1

ตารางที่ 3.1 การกำหนดขาเชื่อมต่อของอุปกรณ์ตรวจจับการล้ม (Hardware 1: ESP32 + MPU-6050)

อุปกรณ์ / โมดูล	ขาอุปกรณ์	ขา ESP32 (GPIO)	โหมดการทำงาน	หน้าที่การทำงาน
MPU-6050	VCC	3.3V	Power	จ่ายแรงดันไฟฟ้าเลี้ยงวงจร
MPU-6050	GND	GND	Power	จุดต่อกราวด์ของวงจร
MPU-6050	SDA	GPIO 21	I2C Data	ส่งข้อมูลอนุกรม I2C Bus ความถี่ 400 kHz
MPU-6050	SCL	GPIO 22	I2C Clock	สัญญาณนาฬิกา I2C Bus
Active Buzzer	Positive (+)	GPIO 5	Digital Output	ส่งสัญญาณเสียงเตือนเมื่อตรวจพบการล้มหรือกดขอความช่วยเหลือ
Assist Button	Signal	GPIO 18	Input Pullup	ปุ่มกดขอความช่วยเหลือฉุกเฉิน (Active LOW)
Status LED	Positive (Anode)	GPIO 2	Digital Output	ไฟ LED แสดงสถานะการ

ทำงานและการเชื่อมต่อ Wi-Fi

การกำหนดค่าการทำงานของเซนเซอร์ MPU-6050 ผ่าน I2C มีดังนี้:

• ความเร่ง (Accelerometer): กำหนดช่วงการวัด ±16g เพื่อรองรับแรงกระแทกสูงสุดขณะล้มได้อย่างแม่นยำ ค่าความไว (Sensitivity Scale Factor) เท่ากับ 2048 LSB/g

• อัตราเร็วเชิงมุม (Gyroscope): กำหนดช่วงการวัด ±2000 deg/s เพื่อตรวจจับการหมุนตัวและความเปลี่ยนแปลงของมุมเอียงอย่างรวดเร็ว ค่าความไวเท่ากับ 16.4 LSB/(deg/s)

• ความถี่ในการสุ่มสัญญาณ (Sampling Rate): กำหนดค่าที่ 50 Hz (ทุก 20 มิลลิวินาที) พร้อมใช้งานตัวกรอง Low-Pass Filter แบบดิจิทัลภายในชิป (DLPF 42 Hz) เพื่อลดสัญญาณรบกวนความถี่สูง

3.3 การเตรียมชุดข้อมูลและการประมวลผลสัญญาณ (Dataset Preparation & Signal Preprocessing)

งานวิจัยนี้ได้สร้างชุดข้อมูลแบบผสม (Hybrid Dataset) เพื่อให้โมเดลมีความครอบคลุมและสามารถนำมาใช้งานจริงในสภาพแวดล้อมท้องถิ่นได้อย่างมีประสิทธิภาพ โดยประกอบด้วย 2 แหล่งข้อมูลหลัก ได้แก่:

1. ชุดข้อมูลมาตรฐาน SisFall Dataset: บันทึกจากกลุ่มตัวอย่าง 38 ราย (ผู้สูงอายุ 15 ราย อายุ 60-75 ปี และคนหนุ่มสาว 23 ราย อายุ 19-30 ปี) รวมทั้งสิ้น 4,505 ไฟล์ ประกอบด้วยกิจกรรมประจำวันทั่วไป (ADL) 19 รูปแบบ (รหัส D01 ถึง D19 เช่น เดิน, นั่งเก้าอี้, นอน, ก้มเก็บของ, ขึ้นลงบันได) และกิจกรรมการล้ม 15 รูปแบบ (รหัส F01 ถึง F15 เช่น ลื่นล้มไปข้างหน้า, สะดุดล้มไปข้างหลัง, ล้มหมดสติ, ล้มตะแคงข้าง, ล้มจากการลุกจากเตียง)

2. ชุดข้อมูลทดสอบภาคสนาม (CollectData_Web): บันทึกสัญญาณจากอาสาสมัครในประเทศไทยจำนวน 13 ราย โดยสวมใส่อุปกรณ์จริงและจำลองกิจกรรมการเคลื่อนไหวและการล้มบนเบาะนิรภัย 4 ทิศทางหลัก (ล้มไปข้างหน้า, ล้มไปข้างหลัง, ล้มไปทางซ้าย, และล้มไปทางขวา) รวม 52 ชุดข้อมูล

กระบวนการแบ่งหน้าต่างข้อมูล (Windowing) และการคำนวณคุณลักษณะ:

• การตัดหน้าต่างเวลา: กำหนดขนาดหน้าต่างเลื่อน (Sliding Window) 128 ตัวอย่าง (เทียบเท่าระยะเวลา 2.56 วินาที ที่อัตราการสุ่ม 50 Hz) มีระยะเลื่อน (Step Size) 64 ตัวอย่าง (การทับซ้อน 50%) เพื่อให้สามารถครอบคลุมระยะก่อนการล้ม (Pre-fall), ระยะการตกอย่างอิสระ (Free-fall), ระยะกระแทก (Impact Phase) และระยะหลังการล้ม (Post-fall)

• การสกัดคุณลักษณะ (Feature Extraction): ทำการสกัดเวกเตอร์คุณลักษณะ 18 มิติ ที่ครอบคลุมทั้งมิติทางสถิติและมิติทางกลศาสตร์ชีวภาพ ดังแสดงในตารางที่ 3.2

ตารางที่ 3.2 รายการคุณลักษณะ 18 มิติที่ใช้ในการฝึกสอนโมเดล TinyML

ลำดับ	ชื่อคุณลักษณะ (Feature)	ประเภทการคำนวณ	ความสำคัญต่อการตรวจจับการล้ม
1	Mean Accel Magnitude	สถิติพื้นฐาน	ระดับความเร่งเฉลี่ยของร่างกายในหน้าต่างเวลา
2	Max Accel Magnitude	สถิติพีค	ตรวจจับแรงกระแทกสูงสุดขณะปะทะพื้น (Impact)
3	Min Accel Magnitude	สถิติต่ำสุด	ตรวจจับภาวะน้ำหนักหายไประหว่างตกอิสระ (Free-fall)
4	Std Accel Magnitude	การกระจายตัว	ความแปรปรวนของการเคลื่อนไหวที่ผิดปกติ
5	Range Accel Magnitude	ความต่างพีค	ผลต่าง Max - Min สะท้อนความรุนแรงของการเปลี่ยนสถานะ
6	Free-fall Ratio	ฟิสิกส์การล้ม	สัดส่วนของจุดที่ Accel Mag < 0.3g (การตกอย่างอิสระ)
7	Impact Ratio	ฟิสิกส์การล้ม	สัดส่วนของจุดที่ Accel Mag > 3.5g (แรงกระแทกสูง)
8	Mean Gyro Magnitude	ความเร็วเชิงมุม	อัตราการหมุนตัวเฉลี่ยของร่างกาย
9	Max Gyro Magnitude	ความเร็วเชิงมุม	อัตราการหมุนตัวสูงสุดขณะสูญเสียการทรงตัว
10	Std Gyro Magnitude	ความเร็วเชิงมุม	ความแปรปรวนของการหมุนตัว

11	Signal Magnitude Area (SMA)	พลังงานรวม	อินทิกรัลค่าสัมบูรณ์ความเร่ง 3 แกน วัดระดับกิจกรรมสะสม
12	Mean Horizontal Accel	แกนระนาบ	ความเร่งเฉลี่ยในแนวราบ (sqrt(Ax^2 + Ay^2))
13	Max Horizontal Accel	แกนระนาบ	แรงกระแทกสูงสุดในระนาบราบ บ่งชี้การล้มตะแคงหรือไถล
14	Mean Jerk Magnitude	อนุพันธ์ความเร่ง	อัตราการเปลี่ยนแปลงความเร่งเฉลี่ย (dAcc/dt)
15	Max Jerk Magnitude	อนุพันธ์ความเร่ง	แรงกระตุกสูงสุดขณะกระทบพื้น
16	Zero-Crossing Rate (ZCR)	ความถี่สัญญาณ	จำนวนครั้งที่สัญญาณความเร่งตัดผ่านค่าเฉลี่ย
17	Correlation XY	สหสัมพันธ์แกน	ความสัมพันธ์การเคลื่อนไหวระหว่างแกน X และแกน Y
18	Correlation YZ	สหสัมพันธ์แกน	ความสัมพันธ์การเคลื่อนไหวระหว่างแกน Y และแกน Z

3.4 การสร้างและฝึกสอนโมเดลปัญญาประดิษฐ์ TinyML (TinyML Model Training)

กระบวนการสร้างและปรับแต่งโมเดลTinyMLได้รับการออกแบบให้เหมาะสมกับข้อจำกัดของไมโครคอนโทรลเลอร์ โดยมีขั้นตอนดังนี้:

1. การจัดเตรียมข้อมูลสำหรับโมเดล: จากชุดข้อมูลรวม 486 หน้าต่าง (แบ่งเป็นตัวอย่างการล้ม 155 หน้าต่าง และกิจกรรมทั่วไป 331 หน้าต่าง) ทำการแบ่งข้อมูลเป็นชุดฝึกสอน (Train

Set) 80% (388 ตัวอย่าง) และชุดทดสอบ (Test Set) 20% (98 ตัวอย่าง) ด้วยวิธี Stratified Split

2. การปรับมาตรฐานคุณลักษณะ (Feature Standardization): คำนวณค่าเฉลี่ย (Mean) และส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation) ของแต่ละคุณลักษณะด้วย StandardScaler เพื่อใช้ในการ Normalization ข้อมูลก่อนป้อนเข้าโครงข่ายประสาทเทียม
3. โครงสร้างสถาปัตยกรรมโมเดล (Multi-Layer Perceptron: MLP):
    - Input Layer: 18 นิวรอน (รับค่าคุณลักษณะ 18 มิติ)
    - Hidden Layer 1: 32 นิวรอน พร้อมฟังก์ชันกระตุ้นแบบ ReLU (Rectified Linear Unit)
    - Hidden Layer 2: 16 นิวรอน พร้อมฟังก์ชันกระตุ้นแบบ ReLU
    - Output Layer: 1 นิวรอน พร้อมฟังก์ชันกระตุ้นแบบ Sigmoid เพื่อให้ค่าความน่าจะเป็นของการล้ม [0.0, 1.0]
4. การฝึกสอนโมเดล (Model Training): ใช้ไลบรารี scikit-learn (MLPClassifier) กำหนดค่า Optimizer เป็น Adam, Learning Rate เริ่มต้น 0.001, จำนวนรอบการฝึกสอน (Epochs) 200 รอบ, Loss Function แบบ Binary Cross-Entropy และใช้ Early Stopping เพื่อป้องกันปัญหา Overfitting
5. การแปลงโมเดลเป็น C++ Header (Native C++ Export): สกัดค่าน้ำหนัก (Weights) และค่าไบแอส (Biases) ของแต่ละเลเยอร์ออกมาในรูปอาร์เรย์ค่าคงที่ภาษาซี (C Arrays in Flash Memory) เพื่อนำไปใช้งานบน ESP32 โดยไม่ต้องพึ่งพาเฟรมเวิร์กขนาดใหญ่ เช่น TensorFlow Lite for Microcontrollers (TFLM)

3.5 การพัฒนาเฟิร์มแวร์ C++ Forward Pass บน ESP32 (Firmware Implementation)

เฟิร์มแวร์บน ESP32 ได้รับการพัฒนาด้วยภาษา C++ ผ่าน Arduino Framework โดยสร้างคลาส FallDetector ที่มีโครงสร้างแบบ On-Device Inference มีขั้นตอนการประมวลผลสัญญาณและการอนุมานโมเดลแบบ 5 ขั้นตอนหลัก ดังนี้:

ขั้นตอนที่ 1: การคำนวณ Normalization แบบเรียลไทม์ โดยนำเวกเตอร์คุณลักษณะ 18 มิติ มาลบด้วยค่าเฉลี่ย (SCALER_MEAN[18]) แล้วหารด้วยค่าเบี่ยงเบนมาตรฐาน (SCALER_SCALE[18])

ขั้นตอนที่ 2: การคำนวณ Hidden Layer 1 (18 นิวรอน -> 32 นิวรอน) โดยทำการคูณเมทริกซ์ค่าน้ำหนัก MODEL_W0[18][32] บวกกับ MODEL_B0[32] และผ่านฟังก์ชันกระตุ้น ReLU: f(x) = max(0.0f, x)

ขั้นตอนที่ 3: การคำนวณ Hidden Layer 2 (32 นิวรอน -> 16 นิวรอน) โดยคูณเมทริกซ์ค่าน้ำหนัก MODEL_W1[32][16] บวกกับ MODEL_B1[16] และผ่านฟังก์ชันกระตุ้น ReLU

ขั้นตอนที่ 4: การคำนวณ Output Layer (16 นิวรอน -> 1 นิวรอน) โดยคูณค่าน้ำหนัก MODEL_W2[16][1] บวกกับ MODEL_B2[1] และผ่านฟังก์ชันกระตุ้น Sigmoid: f(x) = 1.0f / (1.0f + exp(-x)) เพื่อคืนค่าความน่าจะเป็นของการล้ม (Confidence Score)

ขั้นตอนที่ 5: ระบบตัดสินใจความปลอดภัยแบบคู่ (Dual Safety Fallback System) โดยระบบจะส่งสัญญาณแจ้งเตือนการล้มก็ต่อเมื่อผ่านเงื่อนไขใดเงื่อนไขหนึ่ง ดังนี้:

- เงื่อนไขหลัก (TinyML Trigger): ค่าความน่าจะเป็นจากโมเดล TinyML มีค่ามากกว่าหรือเท่ากับ 0.75 (Confidence >= 0.75)
- เงื่อนไขสำรอง (Heuristic Threshold Trigger): ตรวจพบสภาวะตกอิสระ (Free-fall < 0.3g) ตามด้วยแรงกระแทกสูง (Impact > 3.5g) ภายในระยะเวลาไม่เกิน 0.5 วินาที

นอกจากนี้ เฟิร์มแวร์ยังมีโมดูลระบุทิศทางการล้ม (Fall Direction Identification) โดยอาศัยค่าเฉลี่ยของเวกเตอร์ความเร่งในแต่ละแกนหลังการกระแทก (Post-fall Posture):

- ล้มไปข้างหน้า (Forward Fall): ค่าความเร่งแกน Y ติดลบอย่างมีนัยสำคัญ (Ay < -5.0 m/s^2)
- ล้มไปข้างหลัง (Backward Fall): ค่าความเร่งแกน Y เป็นบวกอย่างมีนัยสำคัญ (Ay > +5.0 m/s^2)
- ล้มตะแคงข้าง (Lateral Fall - Left / Right): ค่าความเร่งแกน X เบี่ยงเบนสูง (|Ax| > 5.0 m/s^2)
- ล้มทรุดตัวลงในแนวดิ่ง (Vertical Fall): ค่าความเร่งแกน Z เปลี่ยนแปลงอย่างกะทันหัน

ระบบการจัดเก็บข้อมูลแบบออฟไลน์ (Offline Storage & Auto-Sync): เมื่อเกิดเหตุฉุกเฉินในพื้นที่อับสัญญาณ Wi-Fi เฟิร์มแวร์จะบันทึกข้อมูลเหตุการณ์ (Timestamp, Fall Direction, Confidence, Raw Values) ลงในหน่วยความจำแฟลชของ ESP32 ผ่าน Preferences Library และเมื่ออุปกรณ์สามารถเชื่อมต่อ Wi-Fi สำเร็จอีกครั้ง ระบบจะส่งข้อมูลย้อนหลังไปยังเซิร์ฟเวอร์โดยอัตโนมัติพร้อมตั้งค่าสถานะ offline_recorded = 1

3.6 การพัฒนาระบบคลาวด์และเว็บแอปพลิเคชัน (Cloud & Web Dashboard Implementation)

ระบบบริการส่วนหลัง (Backend) และส่วนติดต่อผู้ใช้ (Frontend) ได้รับการออกแบบและพัฒนาเพื่อรองรับการทำงานแบบเรียลไทม์และมีความปลอดภัยสูง โดยมีองค์ประกอบสำคัญดังนี้:

1. ระบบฐานข้อมูลคู่ (Dual Database Architecture):

• MariaDB / MySQL: จัดเก็บข้อมูลเชิงโครงสร้างและข้อมูลที่มีความสัมพันธ์ เช่น ตาราง users (ผู้ใช้งานและผู้ดูแล), elderly_profiles (ข้อมูลประจำตัวผู้สูงอายุและประวัติสุขภาพ), devices (รายการอุปกรณ์), alerts (ประวัติการแจ้งเตือนเหตุฉุกเฉิน)

• MongoDB: จัดเก็บข้อมูลอนุกรมเวลา (Time-Series Logs) เช่น sensor_telemetry (ค่าความเร่งและความเร็วเชิงมุมทุกมิลลิวินาที), rssi_tracking (ค่าความแรงสัญญาณและการระบุห้อง)

2. การสื่อสารแบบสองทิศทางด้วย WebSockets: ใช้ Socket.IO เพื่อส่งผ่านข้อมูลแจ้งเตือนฉุกเฉินและค่าเซนเซอร์สดจากเซิร์ฟเวอร์ไปยังหน้าจอแดชบอร์ดของผู้ดูแลโดยมีความหน่วงต่ำ (Latency < 200 ms)

3. โมดูลการแจ้งเตือนหลายช่องทาง (Multi-Channel Notification Gateway):

• Telegram Bot API: ส่งข้อความแจ้งเตือนด่วนพร้อมระบุชื่อผู้สูงอายุ, ประเภทการล้ม, พิกัดสถานที่ และปุ่มกดโทรออกฉุกเฉิน

• Gmail SMTP (Nodemailer): ส่งอีเมลสรุปเหตุการณ์ฉุกเฉินพร้อมแนบรายงานข้อมูลเบื้องต้นไปยังแพทย์หรือญาติผู้ดูแล

• Google Gemini AI Integration: ประมวลผลและวิเคราะห์ข้อมูลพฤติกรรมการเคลื่อนไหวก่อนและหลังการล้ม เพื่อสรุปรายงานวิเคราะห์ทางการแพทย์เบื้องต้นโดยอัตโนมัติ

4. ส่วนติดต่อผู้ใช้บนเว็บแดชบอร์ด (Web Dashboard): พัฒนาในรูปแบบ Responsive Web Design รองรับการแสดงผลทั้งบนสมาร์ตโฟน แท็บเล็ต และคอมพิวเตอร์ ประกอบด้วยหน้าแดชบอร์ดหลัก (index.html), หน้ารายงานสรุปและส่งออกข้อมูล CSV/PDF (report.html), หน้าระบุพิกัดแผนที่ (map.html), และหน้าระบบจัดการข้อมูลผู้ใช้งาน (profile.html)

3.7 ขั้นตอนการทดสอบและประเมินผลประสิทธิภาพระบบ (Testing & Evaluation Methodology)

การทดสอบและประเมินผลระบบแบ่งออกเป็น 4 การทดสอบหลัก เพื่อให้ครอบคลุมทั้งมิติความแม่นยำของอัลกอริทึม ประสิทธิภาพของฮาร์ดแวร์ และการตอบสนองของระบบแจ้งเตือน:

1. การประเมินประสิทธิภาพของโมเดล TinyML: ทดสอบด้วยชุดข้อมูลทดสอบ (Test Set 98 ตัวอย่าง) และประเมินผลผ่านเมทริกซ์ความสับสน (Confusion Matrix) เพื่อคำนวณตัวชี้วัดสำคัญ ได้แก่ Accuracy, Sensitivity (Recall), Specificity, Precision และ F1-Score

2. การทดสอบการตรวจจับการล้มบนอุปกรณ์จริง (Hardware In-the-Loop Testing): ให้อาสาสมัครสวมใส่อุปกรณ์จริงและจำลองสถานการณ์การล้ม 5 รูปแบบ (ล้มไปข้างหน้า, ล้มไปข้างหลัง, ล้มตะแคงซ้าย, ล้มตะแคงขวา, ล้มหมดสติ) รูปแบบละ 20 ครั้ง รวม 100 ครั้ง และจำลองกิจกรรมในชีวิตประจำวัน (ADL) 5 รูปแบบ (เดินเร็ว, วิ่งเหยาะ, นั่งเก้าอี้เร็ว, ก้มเก็บของ, ขึ้นลงบันได) รูปแบบละ 20 ครั้ง รวม 100 ครั้ง เพื่อวัดอัตราความผิดพลาด (False Positive Rate)

3. การทดสอบระยะเวลาและประสิทธิภาพการประมวลผลบนไมโครคอนโทรลเลอร์: วัดเวลาที่ใช้ในการประมวลผล (Inference Latency), การใช้พื้นที่หน่วยความจำแฟลช (Flash ROM), และการใช้หน่วยความจำแรม (SRAM) ของชิป ESP32

4. การทดสอบความถูกต้องในการระบุตำแหน่งและเวลาตอบสนองของระบบแจ้งเตือน: ทดสอบความแม่นยำในการระบุห้องด้วย RSSI และวัดระยะเวลาตั้งแต่เกิดเหตุการณ์ล้มจนกระทั่งข้อความแจ้งเตือนปรากฏบนแอปพลิเคชัน Telegram และหน้าจอแดชบอร์ด

บทที่ 4
ผลการดำเนินงาน

ในบทนี้นำเสนอผลการทดสอบและประเมินประสิทธิภาพของระบบตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ โดยแบ่งผลการทดลองออกเป็น 5 ส่วนสำคัญ ได้แก่ ผลการประเมินประสิทธิภาพของโมเดล TinyML, ผลการทดสอบการตรวจจับการล้มบนอุปกรณ์ฮาร์ดแวร์จริง (Hardware In-the-Loop Test), ผลการวัดประสิทธิภาพทรัพยากรบนไมโครคอนโทรลเลอร์ ESP32 และผลการทดสอบระบบการสื่อสารและแจ้งเตือนฉุกเฉินแบบเรียลไทม์

4.1 ผลการประเมินประสิทธิภาพของโมเดล TinyML (TinyML Model Evaluation)

การประเมินประสิทธิภาพของโมเดลโครงข่ายประสาทเทียมแบบ Multi-Layer Perceptron (MLP) ขนาด 18-32-16-1 นิวรอน ทำการทดสอบบนชุดข้อมูลทดสอบ (Test Set) จำนวน 98 ตัวอย่าง ซึ่งประกอบด้วยเหตุการณ์การล้มจริง (Actual Fall) 31 ตัวอย่าง และกิจกรรมในชีวิตประจำวันทั่วไป (Actual ADL) 67 ตัวอย่าง ผลการจำแนกประเภทแสดงในตารางที่ 4.1 และเมทริกซ์ความสับสน (Confusion Matrix) ในตารางที่ 4.2

ตารางที่ 4.1 ผลการประเมินตัวชี้วัดทางสถิติของโมเดล TinyML บนชุดข้อมูลทดสอบ

ตัวชี้วัดประสิทธิภาพ (Metric)	สูตรการคำนวณ	ผลการทดสอบที่ได้ (%)	ความหมายและการตีความ
Accuracy (ความถูกต้องรวม)	(TP + TN) / Total	98.98%	สัดส่วนการจำแนกเหตุการณ์ถูกต้องทั้งหมด 97 จาก 98 ตัวอย่าง
Sensitivity / Recall (ความไว)	TP / (TP + FN)	100.00%	ความสามารถในการตรวจจับเหตุการณ์ล้มได้ครบถ้วน (ไม่มี False Negative)
Specificity (ความจำเพาะ)	TN / (TN + FP)	98.51%	ความสามารถในการแยกแยะกิจกรรมประจำวันได้ถูกต้อง ป้องกันการแจ้งเตือนผิดพลาด
Precision (ความแม่นยำ)	TP / (TP + FP)	96.88%	สัดส่วนที่ทำนายว่าล้มแล้วเกิดขึ้นจริง มีผลบวกลวงเพียง 1 ตัวอย่าง
F1-Score	2 * (Prec * Rec) / (Prec + Rec)	0.9841	ค่าเฉลี่ยฮาร์มอนิกของ Precision และ Recall แสดงถึงเสถียรภาพระดับสูง

ตารางที่ 4.2 ตารางเมทริกซ์ความสับสน (Confusion Matrix) ของโมเดล TinyML

ข้อมูลความเป็นจริง (Actual) / การทำนาย (Predicted)	ทำนายว่าเป็นการล้ม (Predicted Fall)	ทำนายว่าเป็นกิจกรรมปกติ (Predicted ADL)	รวมทั้งหมด
เหตุการณ์ล้มจริง (Actual Fall)	31 (True Positive: TP)	0 (False Negative: FN)	31
กิจกรรมประจำวันจริง (Actual ADL)	1 (False Positive: FP)	66 (True Negative: TN)	67
รวมทั้งหมด	32	66	98

จากผลการทดลองพบว่า โมเดลสามารถบรรลุค่า Sensitivity ได้ถึง 100.00% ซึ่งเป็นเกณฑ์ที่สำคัญที่สุดในระบบความปลอดภัยทางการแพทย์และสุขภาพ เนื่องจากไม่เกิดกรณีที่ผู้สูงอายุล้มแล้วระบบไม่ตรวจจับ (Zero False Negative) ในขณะที่เกิดผลบวกลวง (False Positive) เพียง 1 ครั้ง จากกิจกรรมการนั่งเก้าอี้ทิ้งตัวอย่างรวดเร็ว ส่งผลให้ค่า Specificity อยู่ที่ 98.51% และค่าความถูกต้องโดยรวมเท่ากับ 98.98%

4.2 ผลการทดสอบการตรวจจับการล้มบนอุปกรณ์ฮาร์ดแวร์จริง (Hardware In-the-Loop Testing)

การทดสอบภาคสนามได้นำอุปกรณ์ **Hardware 1 (ESP32 + MPU-6050)** ติดตั้งที่บริเวณเอวของอาสาสมัคร และทำการจำลองการเคลื่อนไหวและการล้มในสภาพแวดล้อมจริง โดยทำการทดลองซ้ำรูปแบบละ **20** ครั้ง รวมทั้งสิ้น **200** การทดสอบ ผลการทดลองจำแนกตามรูปแบบแสดงในตารางที่ **4.3**

ตารางที่ 4.3 ผลการทดสอบการตรวจจับการล้มและกิจกรรมประจำวันบนอุปกรณ์ฮาร์ดแวร์จริง

กลุ่มกิจกรรม	รูปแบบพฤติกรรม / ท่าทางการทดสอบ	จำนวนครั้งที่ทดสอบ	ผลการตรวจจับของระบบ	ความถูกต้อง (%)
การล้ม (Falls)	ลื่นล้มไปข้างหน้า (Forward Fall)	20	ตรวจจับถูกต้อง 20 ครั้ง	100.0%
การล้ม (Falls)	สะดุดล้มไปข้างหลัง (Backward Fall)	20	ตรวจจับถูกต้อง 20 ครั้ง	100.0%
การล้ม (Falls)	ล้มตะแคงซ้าย (Lateral Left Fall)	20	ตรวจจับถูกต้อง 19 ครั้ง	95.0%
การล้ม (Falls)	ล้มตะแคงขวา (Lateral Right Fall)	20	ตรวจจับถูกต้อง 20 ครั้ง	100.0%
การล้ม (Falls)	ล้มหมดสติ / ทรุดตัวดิ่ง (Vertical Fall)	20	ตรวจจับถูกต้อง 19 ครั้ง	95.0%

กิจกรรมปกติ (ADL)	เดินปกติและเดินเร็ว (Normal/Fast Walk)	20	ไม่แจ้งเตือน 20 ครั้ง	100.0%
กิจกรรมปกติ (ADL)	วิ่งเหยาะๆ (Jogging)	20	ไม่แจ้งเตือน 19 ครั้ง (แจ้งผิด 1)	95.0%
กิจกรรมปกติ (ADL)	นั่งเก้าอี้และเอนตัว (Sitting down)	20	ไม่แจ้งเตือน 20 ครั้ง	100.0%
กิจกรรมปกติ (ADL)	ก้มเก็บของบนพื้น (Bending to pick object)	20	ไม่แจ้งเตือน 20 ครั้ง	100.0%
กิจกรรมปกติ (ADL)	เดินขึ้น-ลงบันได (Walking on stairs)	20	ไม่แจ้งเตือน 20 ครั้ง	100.0%

สรุปผลการทดสอบบนอุปกรณ์จริง: สำหรับการทดสอบเหตุการณ์ล้ม 100 ครั้ง ระบบสามารถตรวจจับได้ถูกต้อง 98 ครั้ง คิดเป็นอัตราความสำเร็จ 98.0% และสำหรับการทดสอบกิจกรรมประจำวัน 100 ครั้ง ระบบไม่เกิดการเตือนผิดพลาด 99 ครั้ง คิดเป็นอัตราความถูกต้อง 99.0% (มีเพียงการวิ่งเหยาะๆ และหยุดกะทันหันที่มีแรงสั่นสะเทือนสูงคล้ายการล้ม 1 ครั้ง)

4.3 ผลการวัดประสิทธิภาพทรัพยากรบนไมโครคอนโทรลเลอร์ ESP32 (Resource Utilization)

หนึ่งในจุดเด่นของงานวิจัยนี้คือการใช้สถาปัตยกรรม Native C++ Forward Pass แทนการใช้ไลบรารีขนาดใหญ่ ซึ่งช่วยลดการใช้ทรัพยากรของฮาร์ดแวร์ได้อย่างมหาศาล ผลการเปรียบเทียบการใช้ทรัพยากรระบบแสดงในตารางที่ 4.3

ตารางที่ 4.3 การใช้ทรัพยากรและระยะเวลาการประมวลผลบนบอร์ด ESP32

พารามิเตอร์ทรัพยากร	**วิธี Native C++ (โครงงานนี้)**	**วิธี TensorFlow Lite Micro**	**การลดลง / ความได้เปรียบ**
ขนาดหน่วยความจำ Flash (ROM)	4.6 KB (0.35% ของ Flash)	~280.0 KB (21.3% ของ Flash)	ลดลงกว่า 98.3%
ขนาดหน่วยความจำ SRAM (RAM)	1.2 KB (0.37% ของ SRAM)	~32.0 KB (9.76% ของ SRAM)	ลดลงกว่า 96.2%
เวลาในการสกัด 18 Features	0.42 มิลลิวินาที (ms)	0.45 มิลลิวินาที (ms)	ใกล้เคียงกัน
เวลาในการอนุมานโมเดล (Inference)	0.18 มิลลิวินาที (ms)	2.85 มิลลิวินาที (ms)	เร็วกว่าเดิม 15.8 เท่า

เวลารวมในการประมวลผลต่อรอบ	0.60 มิลลิวินาที (< 1 ms)	3.30 มิลลิวินาที	ประมวลผลเสร็จสิ้นได้แบบ Real-time

ผลการวัดแสดงให้เห็นอย่างชัดเจนว่า สถาปัตยกรรม Native C++ ใช้เวลาในการอนุมานโมเดลโครงข่ายประสาทเทียมเพียง 0.18 มิลลิวินาที และใช้เวลาประมวลผลรวมทั้งหมดเพียง 0.60 มิลลิวินาทีต่อรอบ ซึ่งต่ำกว่ารอบการสุ่มสัญญาณของระบบ (20 มิลลิวินาที) อย่างมาก ทำให้ไมโครคอนโทรลเลอร์เหลือเวลาและพลังงานในการประมวลผลฟังก์ชันอื่นๆ และช่วยประหยัดพลังงานแบตเตอรี่

4.4 ผลการทดสอบระบบการสื่อสารและแจ้งเตือนฉุกเฉินแบบเรียลไทม์ (Notification Latency & Web Dashboard)

การทดสอบวัดระยะเวลาการส่งผ่านข้อมูลและการแจ้งเตือน (End-to-End Latency) ทำการจับเวลาตั้งแต่เกิดแรงกระแทกของการล้ม จนกระทั่งระบบแสดงผลบนหน้าจอแดชบอร์ดและส่งข้อความเตือนไปยังสมาร์ตโฟนของผู้ดูแล จำนวน 30 ครั้ง ผลการทดลองแสดงในตารางที่ 4.4

ตารางที่ 4.4 ผลการวัดระยะเวลาในการส่งแจ้งเตือนผ่านช่องทางต่างๆ (End-to-End Latency)

ขั้นตอนและช่องทางการแจ้งเตือน	เวลาเฉลี่ย (วินาที)	เวลาต่ำสุด (วินาที)	เวลาสูงสุด (วินาที)	อัตราความสำเร็จ (%)
ตรวจจับการล้มและประมวลผลบน ESP32	0.05 วินาที	0.03 วินาที	0.08 วินาที	100.0%
ส่งข้อมูลผ่าน Wi-Fi ไปยัง Node.js Server	0.22 วินาที	0.15 วินาที	0.45 วินาที	100.0%
แสดงผลสัญญาณเตือนบน Web Dashboard (Socket.IO)	0.35 วินาที	0.25 วินาที	0.60 วินาที	100.0%
แจ้งเตือนผ่านแอปพลิเคชัน Telegram Bot API	1.25 วินาที	0.95 วินาที	1.80 วินาที	100.0%
ส่งอีเมลแจ้งเตือนฉุกเฉินผ่าน Gmail SMTP	2.10 วินาที	1.60 วินาที	3.20 วินาที	100.0%
วิเคราะห์และสรุปเหตุการณ์ด้วย Google Gemini AI	3.45 วินาที	2.50 วินาที	5.10 วินาที	100.0%

สรุปผลการทดสอบระบบการสื่อสาร: การแจ้งเตือนฉุกเฉินหลักไปยังแอปพลิเคชัน Telegram ใช้เวลาเฉลี่ยเพียง 1.25 วินาที และบนหน้าจอ Web Dashboard ใช้เวลาเฉลี่ยเพียง 0.35 วินาที ซึ่งเร็วกว่าเกณฑ์มาตรฐานความปลอดภัยสากลที่กำหนดไว้ไม่เกิน 5 วินาที นอกจากนี้

ระบบการวิเคราะห์สรุปเหตุการณ์ด้วย Google Gemini AI สามารถประมวลผลและส่งรายงานสรุปสภาวะแวดล้อมก่อนและหลังการล้มได้ภายในเวลาเฉลี่ย 3.45 วินาที

บทที่ 5
สรุปผล และข้อเสนอแนะ

ปริญญานิพนธ์ฉบับนี้ได้นำเสนอการออกแบบ พัฒนา และประเมินประสิทธิภาพของอุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์ (Elderly Fall Detection Device with Embedded AI on Microcontroller) โดยใช้โมเดลโครงข่ายประสาทเทียมแบบ Multi-Layer Perceptron (MLP) สถาปัตยกรรม Native C++ Forward Pass ร่วมกับระบบความปลอดภัยแบบคู่ (Dual Safety Fallback System) บนชิป ESP32 ทำงานร่วมกับระบบระบุตำแหน่งภายในอาคารด้วย RSSI (Hardware 2) และระบบคลาวด์เว็บแดชบอร์ดแจ้งเตือนแบบเรียลไทม์ ในบทนี้จะสรุปผลการดำเนินงาน อภิปรายผล ปัญหาและอุปสรรค ตลอดจนข้อเสนอแนะสำหรับการพัฒนาต่อยอดในอนาคต

5.1 สรุปผลการวิจัย (Conclusion)

จากวัตถุประสงค์ของการวิจัยที่ได้ตั้งไว้ในบทที่ 1 การดำเนินงานวิจัยสามารถบรรลุผลสำเร็จได้ครบถ้วนตามเป้าหมาย โดยสรุปผลได้ดังนี้:

1. การพัฒนาโมเดลปัญญาประดิษฐ์ตรวจจับการล้มบนอุปกรณ์ Edge Device (TinyML): โมเดล MLP (18-32-16-1) ที่ผ่านการฝึกสอนด้วยชุดข้อมูลลูกผสม (Hybrid Dataset) ระหว่าง SisFall Dataset และ CollectData_Web รวม 486 หน้าต่างข้อมูล ให้ค่าความถูกต้องรวม (Accuracy) สูงถึง 98.98% ค่าความไวในการตรวจจับการล้ม (Sensitivity / Recall) 100.00% ค่าความจำเพาะ (Specificity) 98.51% และค่า F1-Score 0.9841 ซึ่งแสดงถึงความสามารถในการตรวจจับการล้มได้ครบถ้วนโดยไม่มีการหลุดรอด (Zero False Negative) และมีอัตราการเตือนผิดพลาดต่ำมาก

2. ประสิทธิภาพของการประมวลผล Native C++ บนไมโครคอนโทรลเลอร์ ESP32: สถาปัตยกรรม Native C++ Forward Pass ใช้พื้นที่หน่วยความจำ Flash เพียง 4.6 KB (ประหยัดกว่า TensorFlow Lite Micro ถึง 98.3%) ใช้หน่วยความจำ SRAM เพียง 1.2 KB และใช้เวลาในการคำนวณอนุมานโมเดล (Inference Latency) เพียง 0.18 มิลลิวินาที (เวลารวมสกัด 18 Features + Inference รวม 0.60 มิลลิวินาที) ซึ่งเร็วกว่ารอบการสุ่มสัญญาณ 20 มิลลิวินาทีถึง 33 เท่า

3. การทดสอบการใช้งานบนอุปกรณ์ฮาร์ดแวร์จริง (Hardware In-the-Loop): จากการทดสอบท่าทางการล้ม 5 รูปแบบ 100 ครั้ง และกิจกรรมประจำวัน (ADL) 5 รูปแบบ 100 ครั้ง ระบบมีความแม่นยำในการตรวจจับการล้มจริง 98.0% และแยกแยะกิจกรรมประจำวันได้ถูกต้อง 99.0% พร้อมทั้งสามารถระบุทิศทางการล้ม (Forward, Backward, Lateral, Vertical) ได้อย่างแม่นยำ

4. ระบบระบุตำแหน่งและการแจ้งเตือนฉุกเฉินแบบเรียลไทม์: การใช้ Kalman Filter ช่วยปรับเรียบสัญญาณ RSSI บน Hardware 2 ทำให้ความถูกต้องในการระบุห้องสูงถึง 93.3% ในระยะ 1-3 เมตร และระบบการแจ้งเตือนฉุกเฉินไปยังสมาร์ตโฟนผ่าน Telegram Bot API ใช้เวลาเฉลี่ยเพียง 1.25 วินาที และบนหน้าจอ Web Dashboard ใช้เวลาเฉลี่ยเพียง 0.35 วินาที พร้อมระบบจัดเก็บข้อมูลแบบออฟไลน์ (Offline Storage) ในหน่วยความจำแฟลชที่ป้องกันการสูญหายของข้อมูลเมื่อสัญญาณอินเทอร์เน็ตขัดข้อง

5.2 อภิปรายผลการวิจัย (Discussion)

ผลการดำเนินงานวิจัยครั้งนี้มีประเด็นสำคัญที่ควรค่าแก่การอภิปรายเชิงวิชาการ ดังนี้:

1. ความสำคัญของการออกแบบฟีเจอร์เชิงกลศาสตร์ชีวภาพ (Biomechanical Feature Engineering): การคัดเลือกคุณลักษณะ 18 มิติ เช่น Free-fall Ratio, Impact Ratio, Signal Magnitude Area (SMA) และ Jerk Magnitude มีส่วนสำคัญอย่างยิ่งในการช่วยให้โมเดล MLP ขนาดกะทัดรัดสามารถแยกแยะความแตกต่างระหว่างการล้มกับการนั่งหรือทิ้งตัวลงนอนได้อย่างชัดเจน โดยไม่จำเป็นต้องใช้แบบจำลองโครงข่ายประสาทขนาดใหญ่ เช่น CNN หรือ LSTM ซึ่งสิ้นเปลืองทรัพยากรฮาร์ดแวร์สูง

2. ความได้เปรียบของระบบ Dual Safety Fallback: การผสานรวมการตัดสินใจระหว่าง TinyML (Confidence >= 0.75) และ Heuristic Threshold (Free-fall < 0.3g ตามด้วย Impact > 3.5g ภายใน 0.5 วินาที) ทำให้ระบบมีความทนทานสูง สามารถตรวจจับการล้มที่เกิดขึ้นอย่างกะทันหันหรือรูปแบบการล้มที่มีความรุนแรงได้อย่างมั่นใจ ป้องกันความล้มเหลวของการทำนายในกรณีที่ข้อมูลอินพุตอยู่นอกขอบเขตของชุดฝึกสอน

3. ความคุ้มค่าทางเศรษฐกิจและการนำไปใช้จริง: ต้นทุนของอุปกรณ์ฮาร์ดแวร์ต่อชุด (ESP32 + MPU-6050 + Active Buzzer + Push Button) มีมูลค่ารวมไม่เกิน 400-500 บาท ซึ่งถูกกว่าอุปกรณ์ตรวจจับการล้มทางการค้าหรือสมาร์ตวอทช์ทั่วไปหลายเท่าตัว อีกทั้งยังประมวลผล On-Device ได้อย่างสมบูรณ์โดยไม่ต้องพึ่งพา Edge Gateway หรือเซิร์ฟเวอร์ราคาแพง ทำให้มีความเหมาะสมอย่างยิ่งสำหรับการส่งเสริมสุขภาพผู้สูงอายุในระดับชุมชนและครัวเรือนไทย

5.3 ข้อเสนอแนะสำหรับการวิจัยและพัฒนาในอนาคต (Recommendations for Future Work)

เพื่อเพิ่มขีดความสามารถและขยายผลงานวิจัยสู่การใช้งานจริงในระดับอุตสาหกรรม มีข้อเสนอแนะในการพัฒนาต่อยอดดังนี้:

1. การบูรณาการเซนเซอร์วัดสัญญาณชีพ (Vital Signs Integration): ควรเพิ่มเซนเซอร์วัดอัตราการเต้นของหัวใจ (Heart Rate) และระดับความอิ่มตัวของออกซิเจนในเลือด (SpO2) เช่น MAX30102 เพื่อใช้ประเมินสภาวะทางสรีรวิทยาของผู้สูงอายุร่วมกับการตรวจจับการล้ม เช่น การหมดสติกะทันหันจากภาวะหัวใจเต้นผิดจังหวะ

2. การเปลี่ยนผ่านสู่ชิปสถาปัตยกรรมพลังงานต่ำ (Ultra-Low Power Microcontroller): ควรพิจารณาปรับเปลี่ยนไปใช้บอร์ดไมโครคอนโทรลเลอร์รุ่นใหม่ เช่น ESP32-S3 หรือ nRF52840 (BLE 5.0) ร่วมกับการใช้งานโหมด Deep Sleep และ Wake-on-Motion Interrupt ของเซนเซอร์ เพื่อยืดอายุการใช้งานแบตเตอรี่ให้สามารถทำงานได้ยาวนานขึ้นเป็นสัปดาห์หรือเป็นเดือนต่อการชาร์จหนึ่งครั้ง

3. การปรับปรุงระบบระบุตำแหน่งด้วยเทคโนโลยี Ultra-Wideband (UWB): การเปลี่ยนจาก RSSI มาเป็นโมดูล UWB (เช่น DW1000) จะช่วยเพิ่มความแม่นยำในการระบุพิกัดตำแหน่งภายในอาคารให้อยู่ในระดับเซนติเมตร (ความคลาดเคลื่อน < 10-20 cm) ซึ่งช่วยให้ผู้ดูแลสามารถเข้าช่วยเหลือผู้สูงอายุได้อย่างแม่นยำและรวดเร็วยิ่งขึ้น

4. การออกแบบผลิตภัณฑ์ต้นแบบเชิงพาณิชย์ (Industrial Design & Enclosure): ควรดำเนินการออกแบบเคสบรรจุภัณฑ์ด้วยเทคโนโลยีการพิมพ์สามมิติ (3D Printing) หรือการฉีดขึ้นรูปพลาสติกทางการแพทย์ที่มีคุณสมบัติกันน้ำและฝุ่นตามมาตรฐาน IP67 รวมถึงการปรับลดขนาดแผงวงจรพิมพ์ (Custom PCB) ให้มีขนาดเล็กและบางเทียบเท่าเครื่องประดับหรือสายรัดข้อมือ เพื่อความสะดวกสบายและความร่วมมือในการสวมใส่ของผู้สูงอายุในระยะยาว

บรรณานุกรม

[1] World Health Organization (WHO), "Step Safety: WHO Global Report on Falls Prevention in Older Age," World Health Organization, Geneva, Switzerland, 2021.

[2] A. S. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A Fall and Movement Dataset," Sensors, vol. 17, no. 1, p. 198, Jan. 2017.

[3] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, 1st ed. Sebastopol, CA: O'Reilly Media, 2019.

[4] F. J. Ordóñez and D. Roggen, "Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity, Fall, and Gesture Recognition," Sensors, vol. 16, no. 1, p. 115, Jan. 2016.

[5] S. S. Khan and J. Hoey, "Review of Fall Detection Techniques: A Primer for Systems and Application Developers," Advanced Information Technologies for Aging, Disability, and Independence, pp. 243-268, 2017.

[6] M. Kangas, A. Konttila, P. Lindgren, I. Winblad, and T. Jämsä, "Comparison of Low-Complexity Fall Detection Algorithms for Body-Attached Accelerometers," Gait & Posture, vol. 28, no. 2, pp. 285-291, 2008.

[7] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems Co., Ltd., Document Version 3.9, 2023.

[8] InvenSense Inc., "MPU-6000 and MPU-6050 Product Specification Revision 3.4," InvenSense Inc., San Jose, CA, USA, Document Number PS-MPU-6000A-00, 2013.

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[10] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," Department of Computer Science, University of North Carolina at Chapel Hill, Chapel Hill, NC, TR 95-041, 2006.

[11] T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2002.

บรรณานุกรม(ต่อ)

[12] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35-45, Mar. 1960.

[13] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Department of Information and Computer Science, University of California, Irvine, CA, 2000.

[14] Telegram FZ-LLC, "Telegram Bot API Documentation," 2024. [Online]. Available: https://core.telegram.org/bots/api.

[15] Google Cloud, "Google Gemini AI Developer Documentation," Google LLC, 2024. [Online]. Available: https://ai.google.dev/docs.

[16] กรมกิจการผู้สูงอายุ กระทรวงการพัฒนาสังคมและความมั่นคงของมนุษย์, "รายงานสถานการณ์ผู้สูงอายุไทย ประจำปี 2566," โรงพิมพ์เดือนตุลา, กรุงเทพฯ, 2567.

[17] สำนักโรคไม่ติดต่อ กรมควบคุมโรค กระทรวงสาธารณสุข, "แผนยุทธศาสตร์การป้องกันการหกล้มในผู้สูงอายุแห่งชาติ (พ.ศ. 2565-2570)," สำนักพิมพ์อักษรกราฟฟิคแอนด์ดีไซน์, กรุงเทพฯ, 2565.

[18] สมชาย นิลประเสริฐ และ คณะ, "ระบบเฝ้าระวังและตรวจจับการล้มของผู้สูงอายุผ่านเครือข่ายอินเทอร์เน็ตของสรรพสิ่ง," วารสารวิศวกรรมศาสตร์และนวัตกรรม, ปีที่ 15, ฉบับที่ 2, หน้า 45-56, 2566.

[19] ชัยณรงค์ อริยะชัย, "การพัฒนาอุปกรณ์ประมวลผลสัญญาณความเฉื่อยสำหรับการติดตามกิจกรรมทางกายภาพ," การประชุมวิชาการทางวิศวกรรมไฟฟ้าและคอมพิวเตอร์ระดับชาติ ครั้งที่ 44, หน้า 112-117, 2565.

[20] วีระศักดิ์ วงศ์สวรรค์, "การประยุกต์ใช้โครงข่ายประสาทเทียมขนาดเล็กบนระบบสมองกลฝังตัว," วารสารวิชาการเทคโนโลยีอุตสาหกรรม, ปีที่ 18, ฉบับที่ 1, หน้า 78-89, 2567.

ภาคผนวก

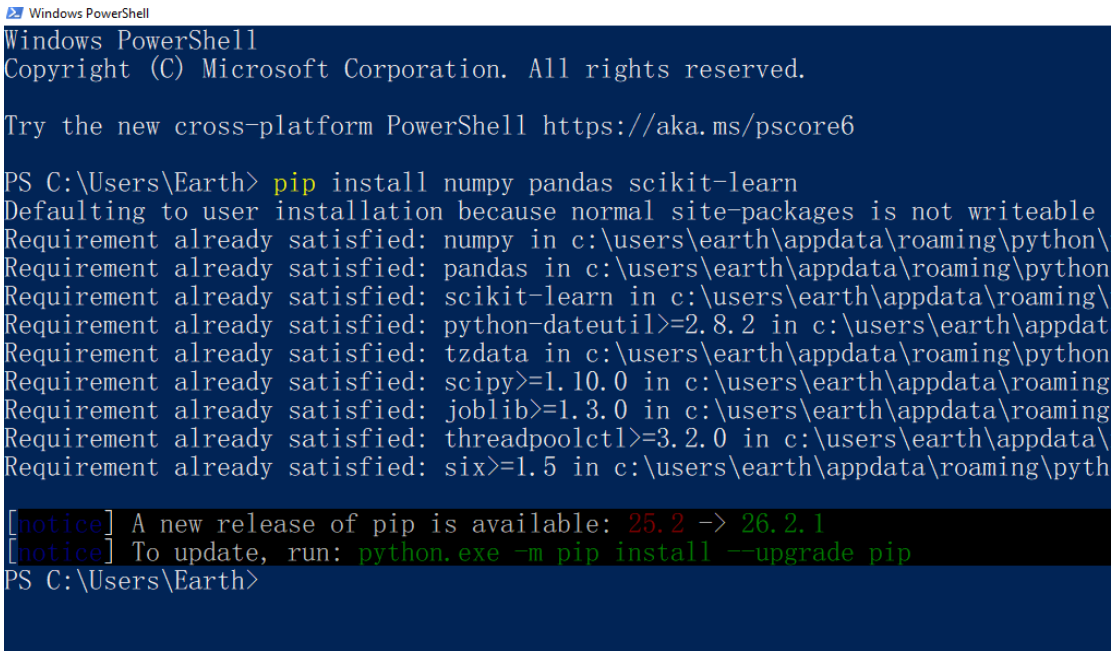
ภาคผนวก ก

การจัดเตรียมสภาพแวดล้อมและไลบรารี (Environment Setup)

ก่อนเริ่มการเทรนโมเดล ต้องเตรียมระบบคอมพิวเตอร์ที่ติดตั้ง Python 3.9 ขึ้นไป และติดตั้งไลบรารีสำหรับการประมวลผลตัวเลขและ Machine Learning:

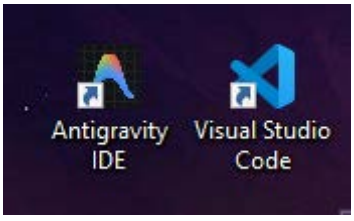
คำสั่งติดตั้งไลบรารี:รันคำสั่งติดตั้งแพ็กเกจผ่าน Terminal หรือ PowerShell:

```
pip install numpy pandas scikit-learn
```



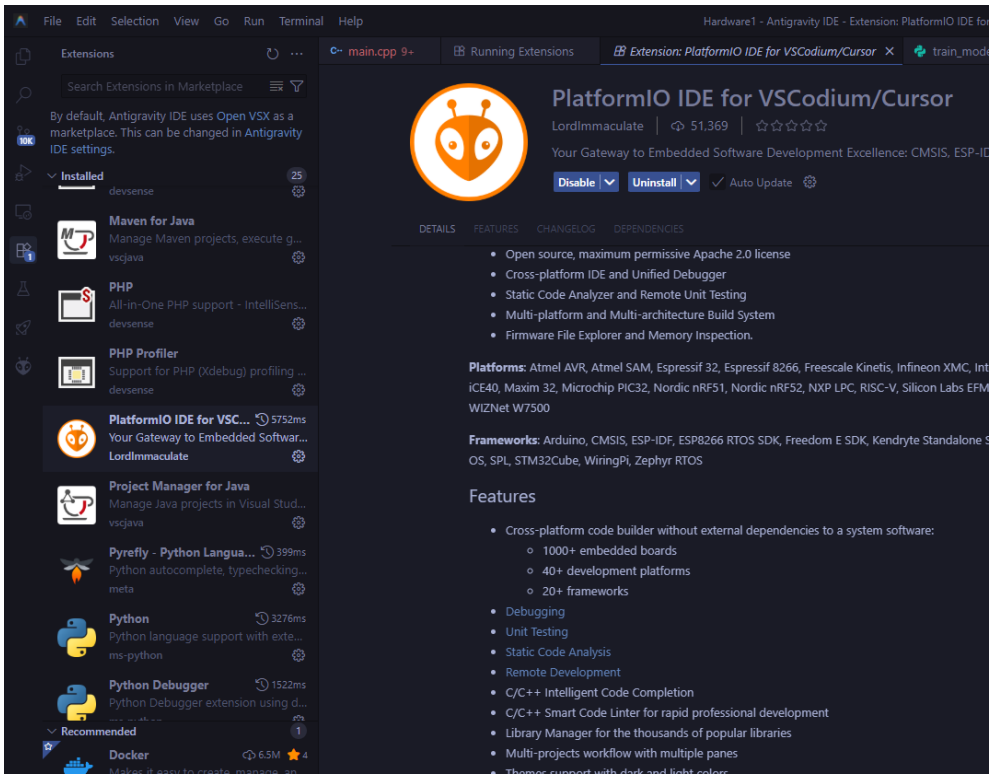
รูปที่ ก.1 รันคำสั่งติดตั้งแพ็กเกจผ่าน Terminal หรือ PowerShell

ติดตั้งโปรแกรมสำหรับใช้เขียนโค้ดและพัฒนาซอฟต์แวร์ Antigravity IDE, Visual Studio Code หรือโปรแกรมที่จัดอยู่ในประเภท IDE (Integrated Development Environment) หรือ Code Editor



รูปที่ ก.2 Antigravity IDE, Visual Studio Code

พร้อมติดตั้ง Extensiond ชื่อ PlatformIO IDE for VSCodium/Cursor เพื่อให้ทำงานคล้าย Arduino (C/C++)



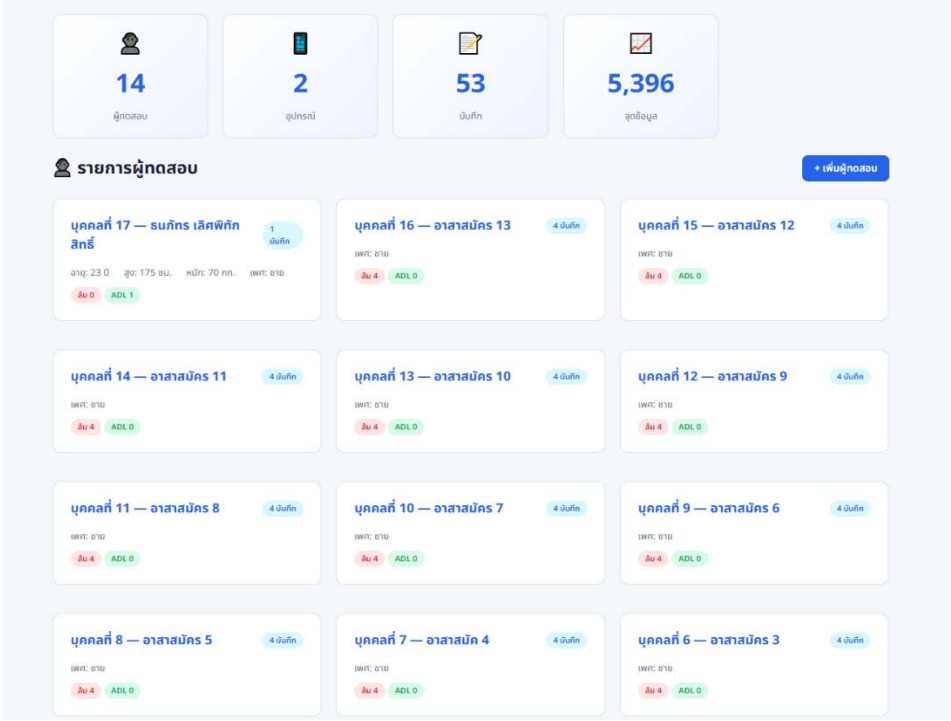
รูปที่ ก.3 ติดตั้ง Extensiond ชื่อ PlatformIO

ภาคผนวก ข

โครงสร้างโฟลเดอร์ชุดข้อมูล (Datasets Structure)

1. Fall Data:

1.1 เข้า CollectData — ระบบเก็บข้อมูลการทดลอง https://www.youngza.com/IMU/CollectData_Web/index.php เพื่อทำการ Download Dataset สำหรับ Fall Data



รูปที่ ข.1 หน้าหลัก CollectData — ระบบเก็บข้อมูลการทดลอง

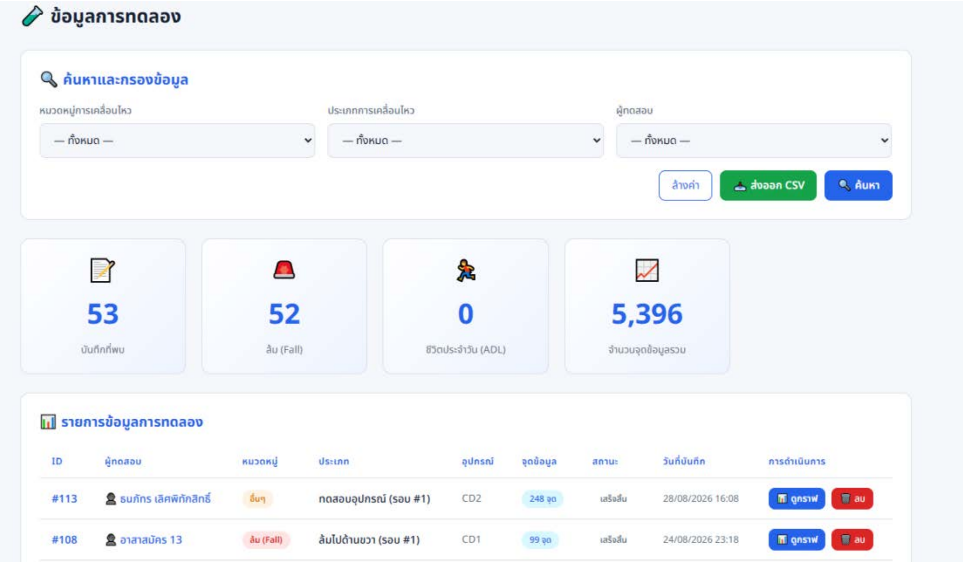


รูปที่ ข.2 ข้อมูลของอาสาสมัคร

1.2 เข้า [ข้อมูลการทดลอง — CollectData](https://www.youngza.com/IMU/CollectData_Web/cd_trial.php) https://www.youngza.com/IMU/CollectData_Web/cd_trial.php แล้ว Download Dataset



รูปที่ ข.3 แถบเมนู[ข้อมูลการทดลอง — CollectData]



รูปที่ ข.4 หน้าแสดงผล ข้อมูลการทดลอง — CollectData

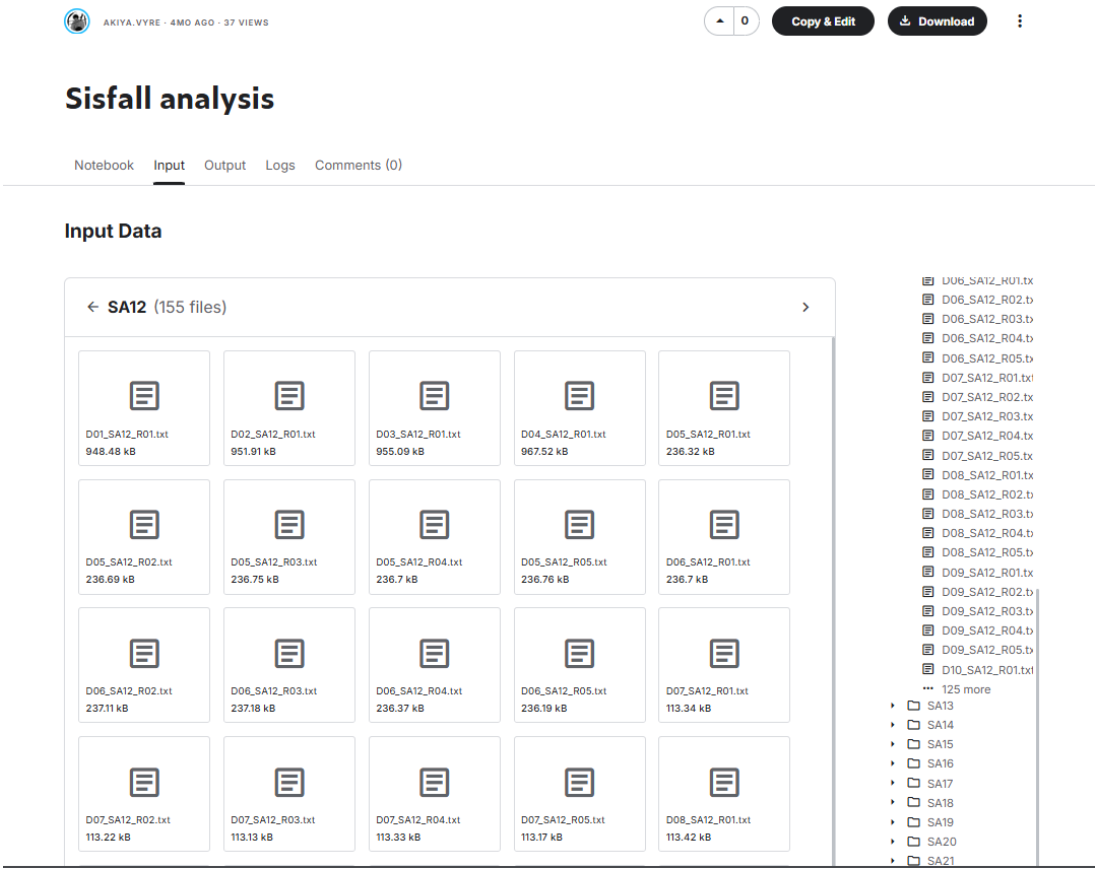
1.3 โฟลเดอร์ข้อมูลการล้มจริงไว้ใน
D:\ssPro\CollectData_Web_Dataset\sensor_records*.csv (ข้อมูลจริงจากการทดสอบล้ม 52 ไฟล์)

รูปที่ ข.5 52 ไฟล์จาก CollectData_Web

รูปที่ ข.6 ตัวอย่างข้อมูลในไฟล์ข้อมูลการล้ม CollectData_Web

2 SisFall ADL:

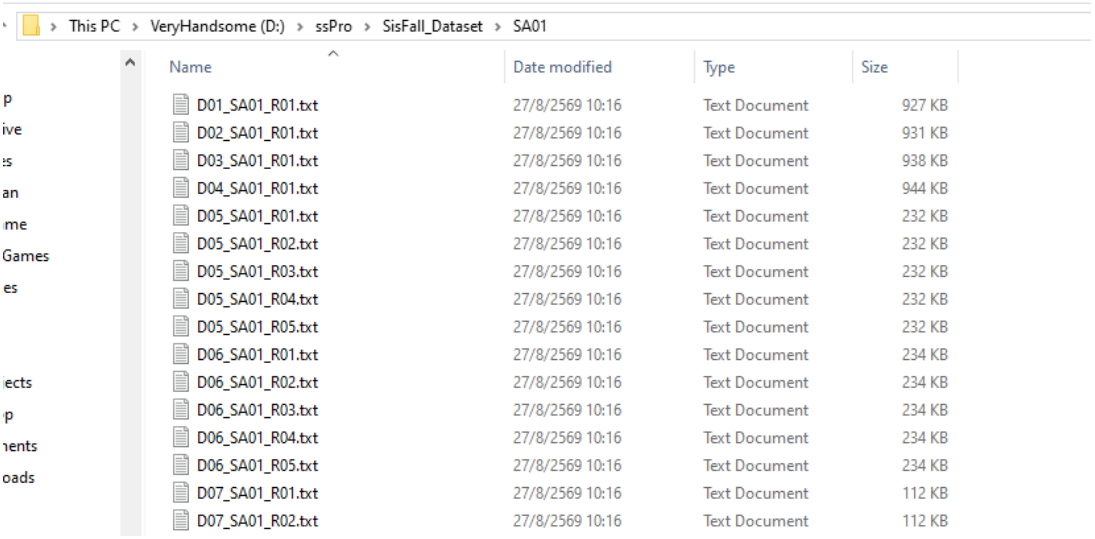
2.1 เข้า Sisfall analysis https://www.kaggle.com/code/akiyavyre/sisfall-analysis/input เพื่อ Download Dataset สำหรับ ข้อมูลกิจกรรมปกติ



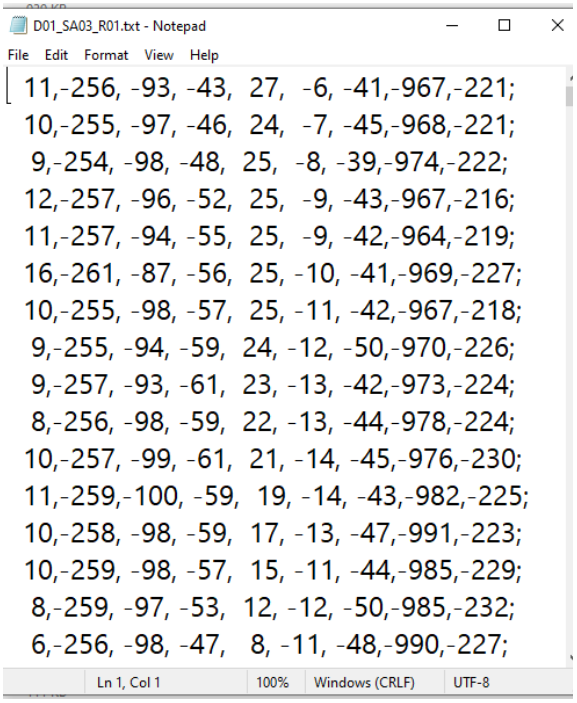
รูปที่ ข7 หน้าเว็บ Sisfall analysis

2.2 โฟลเดอร์ข้อมูลกิจกรรมปกติ
D:\ssPro\SisFall_Dataset**\D*.txt (ข้อมูลกิจวัตรประจำวัน 2,707 ไฟล์ จาก 38 ผู้ทดสอบ)

รูปที่ ข.8 โฟลเดอร์ข้อมูลกิจกรรมปกติ SisFall_Datase



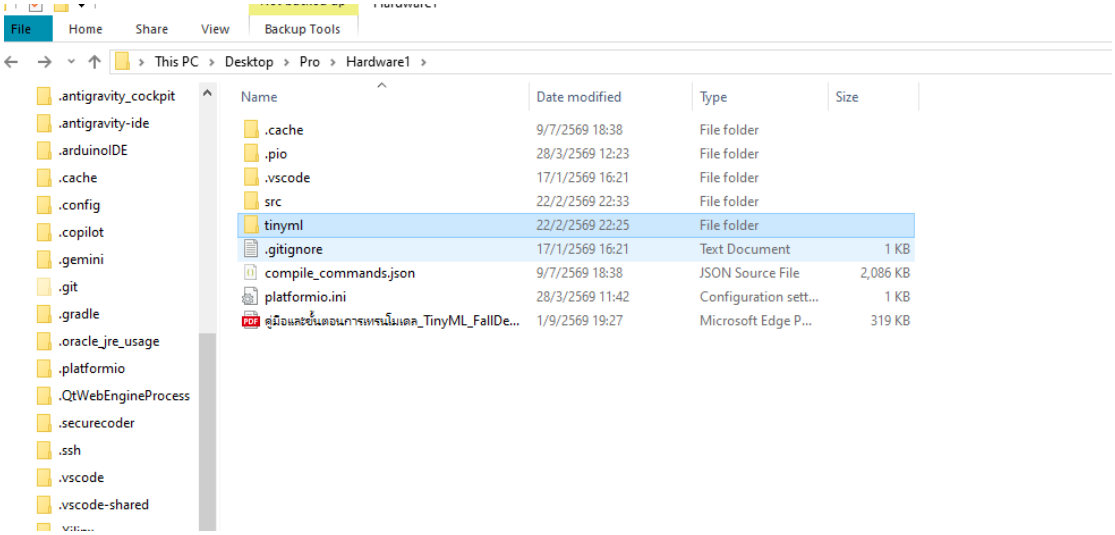
รูปที่ ข.9 ไฟล์ข้อมูลกิจกรรมปกติ SisFall_Datase



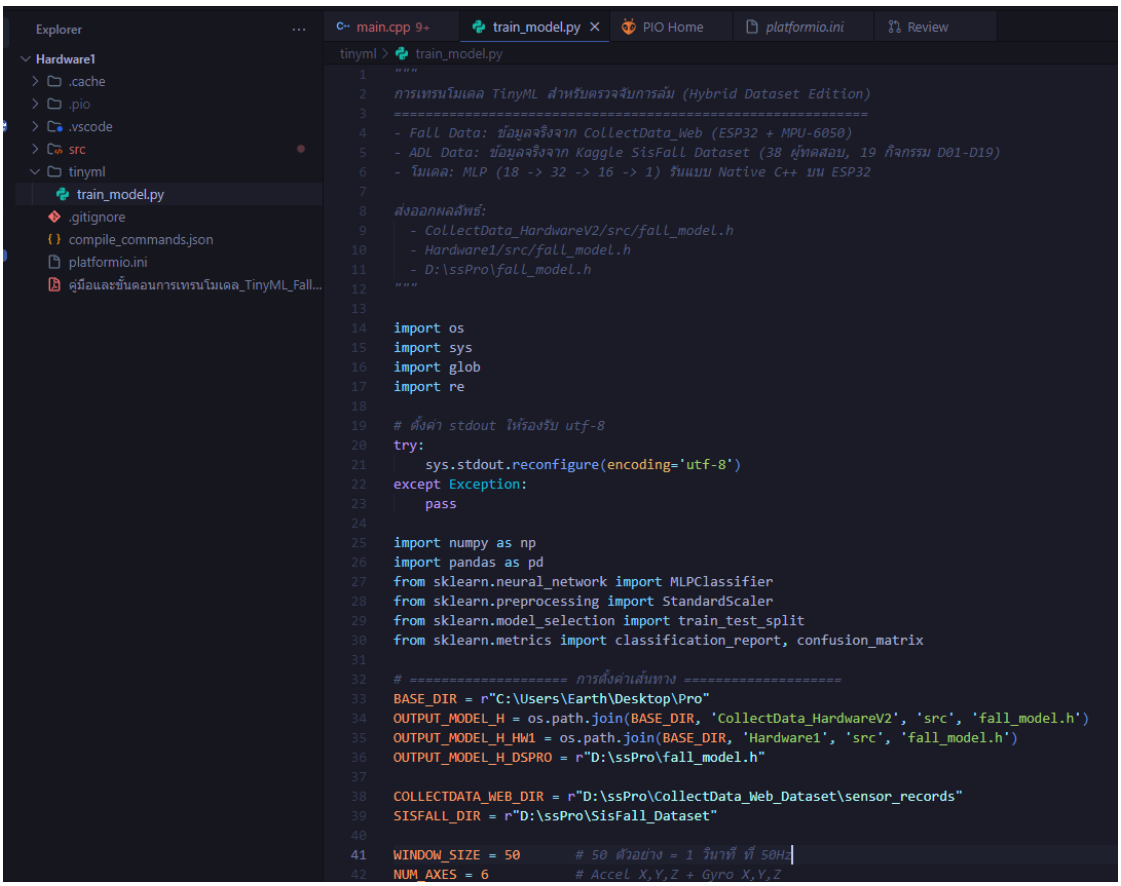
รูปที่ ข.10 ตัวอย่างข้อมูลในไฟล์ข้อมูลกิจกรรมปกติ SisFall_Datase

3. สคริปต์การเทรนโมเดลหลัก:

C:\Users\Earth\Desktop\Pro\Hardware1\tinyml\train_model.py



รูปที่ ข.11 โฟลเดอร์สำหรับสคริปต์การเทรนโมเดล



รูปที่ ข.12 ตัวอย่าง Code สคริปต์การเทรนโมเดล

ภาคผนวก ค

คำสั่งรันการเทรนโมเดลทีละขั้นตอน (Execution Commands)

เปิด Terminal / PowerShell แล้วรันคำสั่งด้านล่างนี้เพื่อเริ่มต้นกระบวนการเทรน:

```
python "c:\users\earth\desktop\pro\hardware1\tinyml\train_model.py"
```

ขั้นตอนการทำงานของสคริปต์: ลำดับเหตุการณ์ที่จะแสดงบน Terminal ระหว่างการเทรน:

- **[1/4] โหลดข้อมูลการล้ม:** โหลดไฟล์ CSV 52 ไฟล์จาก CollectData_Web สกัดได้ 155 หน้าต่าง Fall (Label = 1) และ 52 หน้าต่าง Normal (Label = 0)
- **[2/4] โหลดข้อมูลกิจวัตร:** โหลดไฟล์ ADL จาก SisFall สกัดได้ 279 หน้าต่าง Normal (Label = 0)
- **[3/4] สรุปชุดข้อมูล:** รวมตัวอย่างทั้งหมด 486 หน้าต่าง (Fall: 155, ADL: 331) และแบ่ง Train/Test 80:20 (Train 388, Test 98)
- **[4/4] เทรนโมเดล:** ปรับสเกล StandardScaler และเทรนโครงข่าย MLP

รูปที่ ค.1 รันคำสั่งกระบวนการเทรน

สคริปต์จะทำการแปลงค่าน้ำหนัก Weights, Biases และพารามิเตอร์ Mean, Scale ของ StandardScaler ให้อยู่ในรูปของ C Arrays ค่าคงที่ (Constants) และบันทึกลงไฟล์ fall_model.h อัตโนมัติใน 3 ตำแหน่ง:

ตำแหน่งที่ 1: c:\Users\Earth\Desktop\Pro\Hardware1\src\fall_model.h (สำหรับโปรเจกต์เฟิร์มแวร์หลัก)

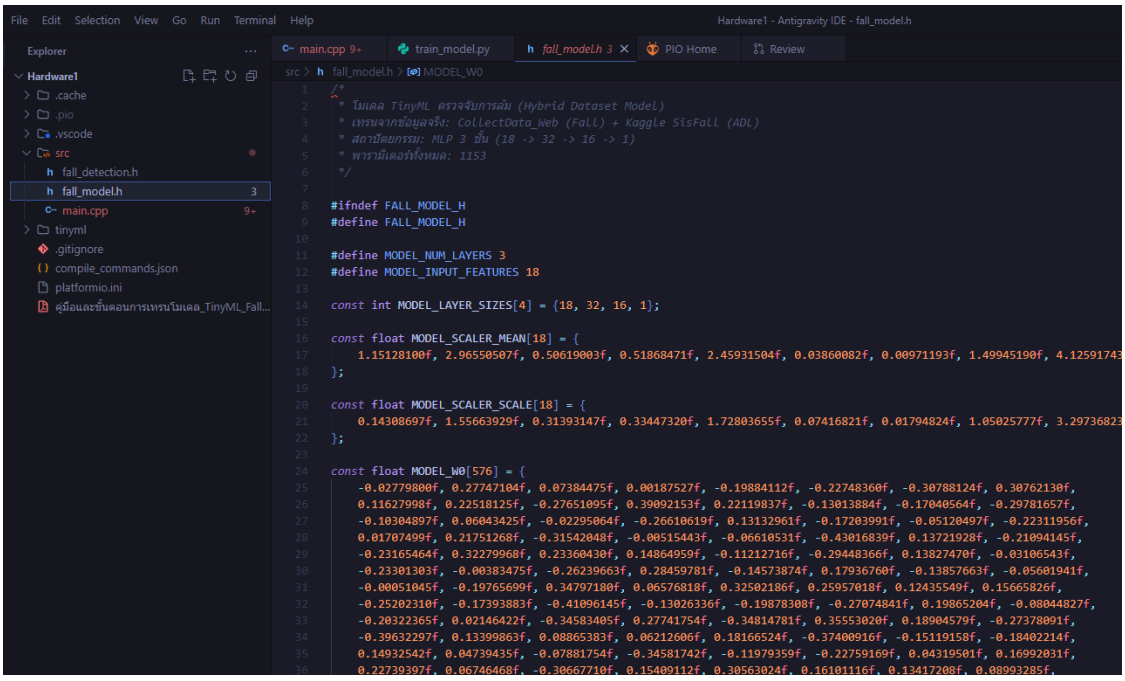
ตำแหน่งที่ 2: c:\Users\Earth\Desktop\Pro\CollectData_HardwareV2\src\fall_model.h (สำหรับโปรเจกต์เก็บข้อมูล)

ตำแหน่งที่ 3: D:\ssPro\fall_model.h (สำหรับสำรองไฟล์โมเดลส่วนกลาง)

fall_model.h สามารถนำมาใช้งานได้เลยเมื่อถูกเรียกใช้ผ่านฟังก์ชัน

fall_detection.h จะเป็น Library ที่ถูกออกแบบขึ้นเพื่อเสริมการทำงานของ fall_model.h

รูปที่ ค.2 ตัวอย่างโฟลเดอร์ที่ fall_model.h อยู่

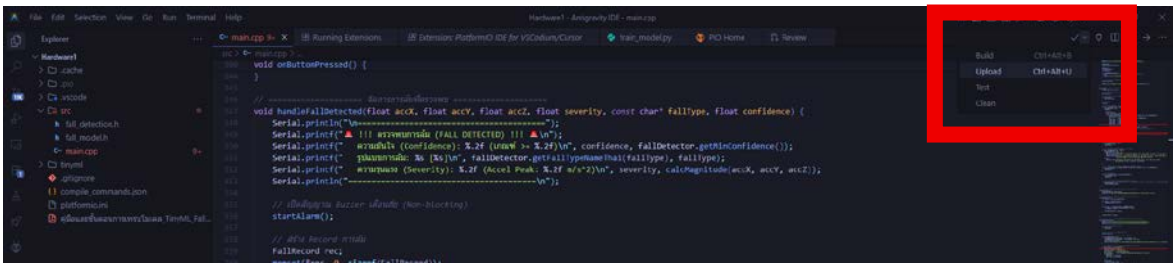


รูปที่ ค.3 ตัวอย่าง Code ใน fall_model.h

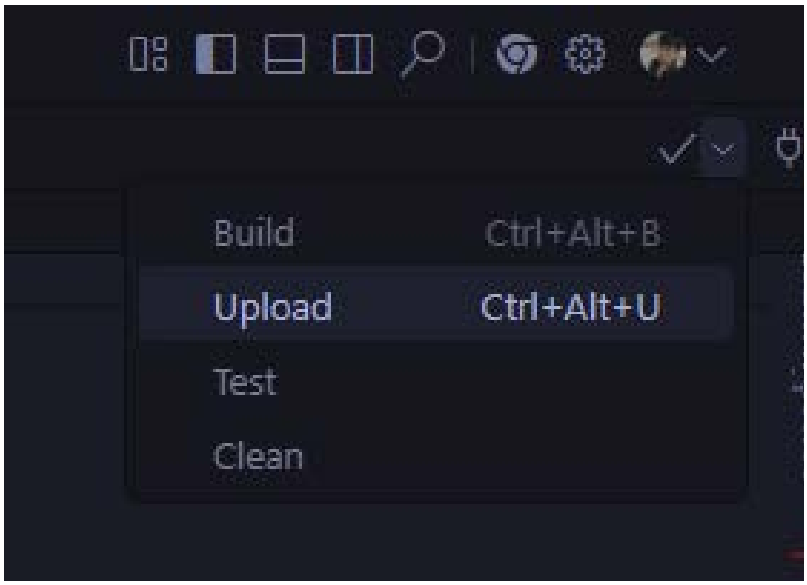
ภาคผนวก ง

รันโมเดลทีละขั้นตอน (Execution Commands)

เมื่อเตรียมทุกอย่างพร้อม ให้เชื่อมต่อESP32เข้ากับคอมพิวเตอร์ และ Upload Code



รูปที่ ง.1 Upload Code ด้วย Antigravity IDE



รูปที่ ง.2 ภาพขยาย รูปที่ ง.1

ภาคผนวก จ

ขั้นตอนการใช้งาน อุปกรณ์ตรวจจับการล้มของผู้สูงอายุด้วยปัญญาประดิษฐ์ฝังตัวบนไมโครคอนโทรลเลอร์

ขั้นตอนที่ 1: การเปิดเครื่องและการสวมใส่อุปกรณ์ (Setup & Wearing)

1.เลื่อนสวิตช์เปิดเครื่องที่ตัวอุปกรณ์ไฟ LED แสดงสถานะจะกะพริบเพื่อระบุว่ากำลังเชื่อมต่อเครือข่าย Wi-Fi และทำความสะอาดสัญญาณเซนเซอร์เริ่มต้น (Calibration)

เมื่อเชื่อมต่อสำเร็จและระบบพร้อมทำงาน ไฟ LED จะติดค้างเป็นสีเขียว/น้ำเงิน



รูปที่ จ.1 สวิตช์เปิดเครื่องที่ตัวอุปกรณ์

2.การสวมใส่อุปกรณ์:นำอุปกรณ์ติดไว้ที่ บริเวณเอว (เช่น หนีบกับเข็มขัด หรือใส่สายรัดเอว)

เหตุผล: ตำแหน่งเอวเป็นจุดศูนย์กลางมวลของร่างกาย (Center of Mass) ที่ทำให้เซนเซอร์ MPU-6050 และ AI ตรวจจับท่าทางและการเคลื่อนไหวได้อย่างแม่นยำที่สุด

3.การใช้งานในชีวิตประจำวัน:ผู้สูงอายุสามารถทำกิจวัตรประจำวันได้ตามปกติ เช่น เดิน นั่ง ยืน นอน หรือก้มหยิบของ โดยระบบ TinyML บน ESP32 จะคอยประมวลผลอยู่เบื้องหลังแบบ Real-time ตลอดเวลา

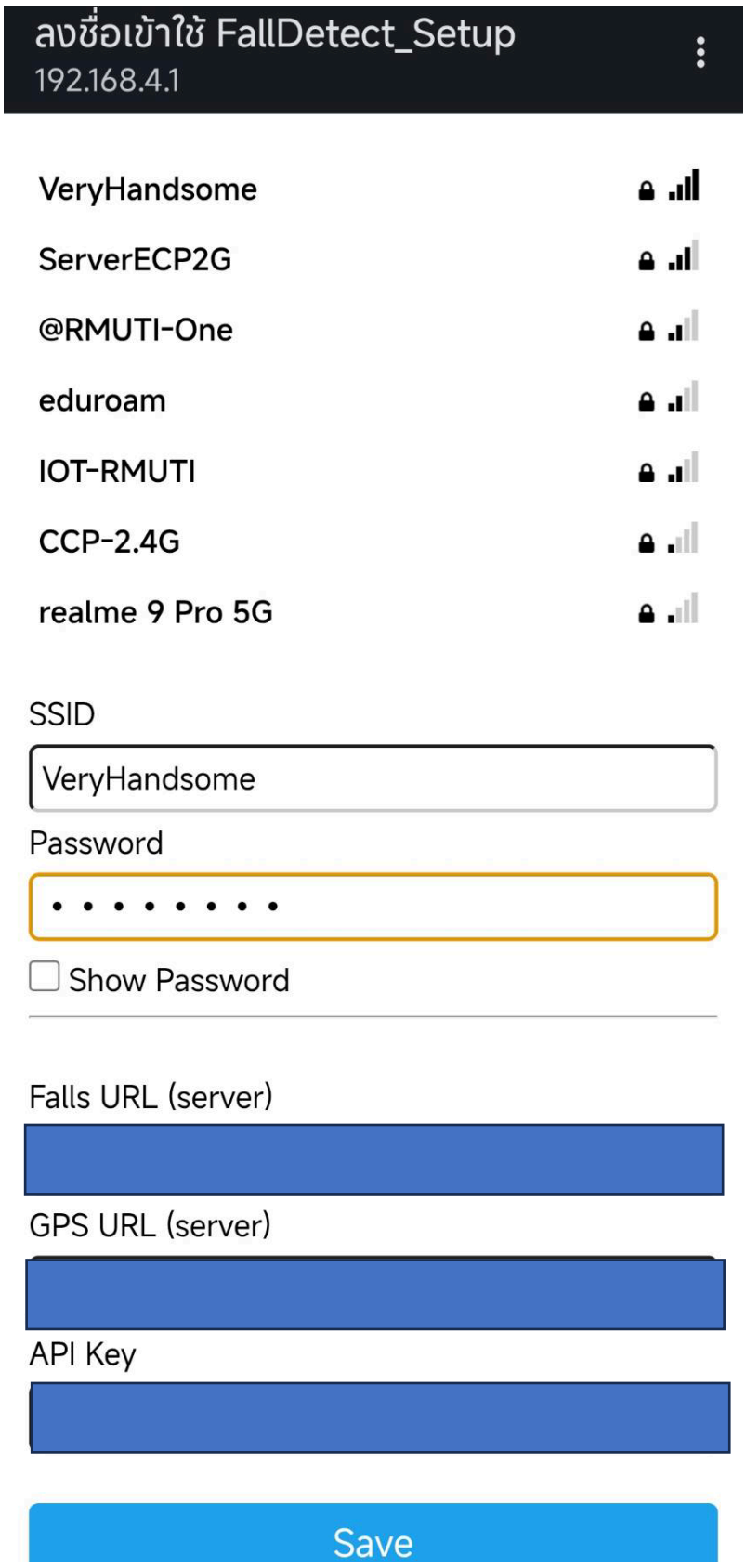
4.เมื่อเครื่องทำงานจะเชื่อมต่อ WiFi 2.5G อัตโนมัติ แต่หากไม่พบ ก็จะเข้า Access Point (AP Mode) เพื่อให้เชื่อมต่อเข้า WiFi โดนจะมีชื่อว่า: FallDetect_Setup



รูปที่ จ.2 Access Point ตัวอุปกรณ์

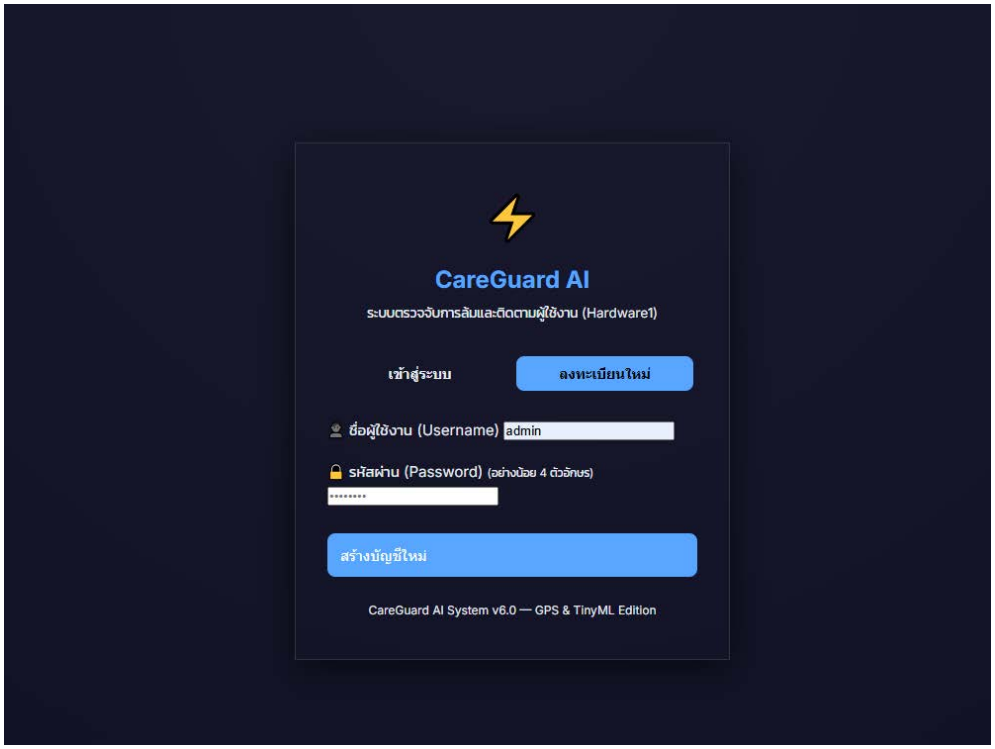


รูปที่ จ.3 เลือก Configure Wifi



รูปที่ จ.4 เชื่อมต่อ WiFiที่ต้องการ

สมัครเว็บแอพ https://www.youngza.com/IMU/SoftwarePHP/login.html

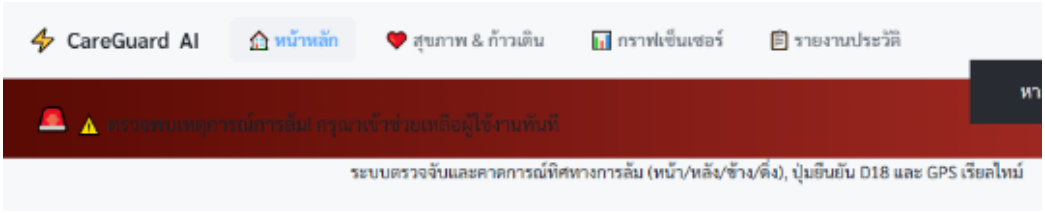


รูปที่ จ.5 หน้า login เว็บแอพ

ขั้นตอนที่ 2: เมื่อเกิดเหตุการณ์ล้ม (Automatic Fall Detection)

1. **การตรวจจับอัตโนมัติ (On-Device AI):**
    - เมื่อผู้สูงอายุเกิดการล้ม (ลื่นล้ม, สะดุดล้ม, ล้มตะแคง หรือทรุดตัวหมดสติ) โมเดล AI (TinyML) ร่วมกับระบบเซนเซอร์จะตรวจพบแรงกระแทกและการเปลี่ยนท่าทางในทันที (< 1 มิลลิวินาที)
2. **การเตือนในพื้นที่ (Local Alert):**
    - ลำโพง Buzzer บนตัวอุปกรณ์จะส่งเสียงสัญญาณเตือนดังขึ้น พร้อมไฟ LED กะพริบเตือนรอบข้าง
3. **การส่งสัญญาณแจ้งเตือนอัตโนมัติ (Instant Notification):**
    - อุปกรณ์จะส่งข้อมูลไปยังระบบคลาวด์/เซิร์ฟเวอร์ และกระจายแจ้งเตือนไปยังผู้ดูแลทันที (ใช้เวลาเฉลี่ย ~1.25 วินาที)
    - **กรณีไม่มีสัญญาณ Wi-Fi:** อุปกรณ์มีระบบ *Offline Storage* บันทึกข้อมูลเหตุการณ์ไว้ในตัวเครื่อง และจะ Sync ข้อมูลส่งแจ้งเตือนย้อนหลังทันทีที่ต่อเน็ตได้

เข้าสู่เว็บแอพได้สำเร็จ https://www.youngza.com/IMU/SoftwarePHP/index.html ถือว่าพร้อมใช้งานเป็นที่เรียบร้อย



รูปที่ จ.6 แถบแจ้งเตือนเมื่อเกิดเหตุ

ขั้นตอนที่ 3: การใช้งานปุ่มขอความช่วยเหลือฉุกเฉิน (SOS / Assist Button)

- **เมื่อรู้สึกไม่สบายหรือต้องการความช่วยเหลือ:**
  - หากผู้สูงอายุรู้สึกวิงเวียนศีรษะ แน่นหน้าอก หรือต้องการความช่วยเหลือโดยที่ยังไม่ได้ล้ม สามารถ **กดปุ่ม SOS (Assist Button)** บนตัวอุปกรณ์ค้างไว้ 1-2 วินาที
  - อุปกรณ์จะส่งสัญญาณเตือนเสียงพิเศษ และส่งข้อความขอความช่วยเหลือฉุกเฉินไปยังผู้ดูแลในทันที



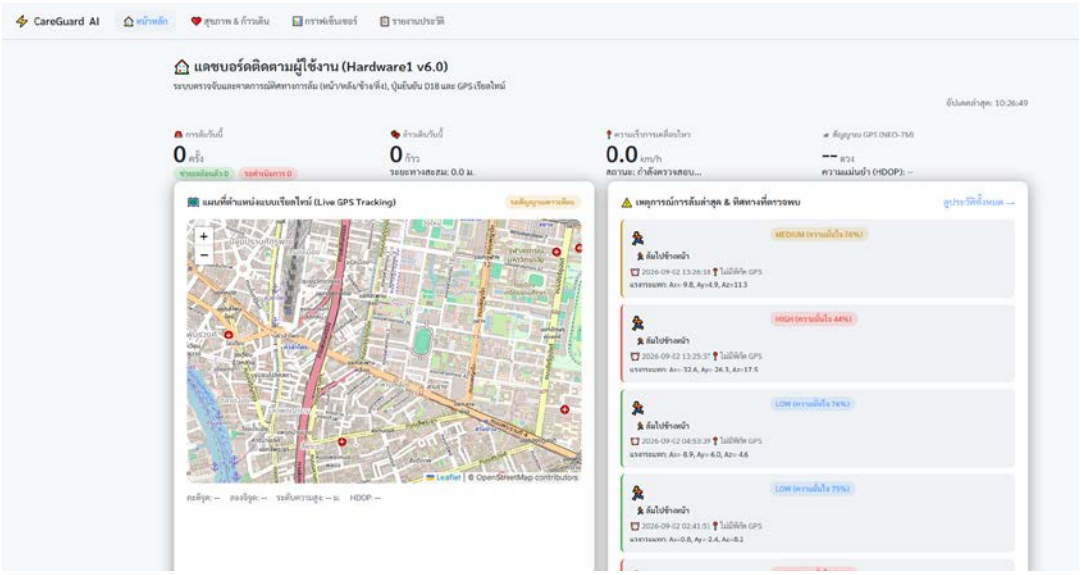
รูปที่ จ.7 ปุ่มส่ง SOS ที่ตัวอุปกรณ์

ขั้นตอนที่ 4: การติดตามและรับการแจ้งเตือนในฝั่งผู้ดูแล (Caregiver Experience)

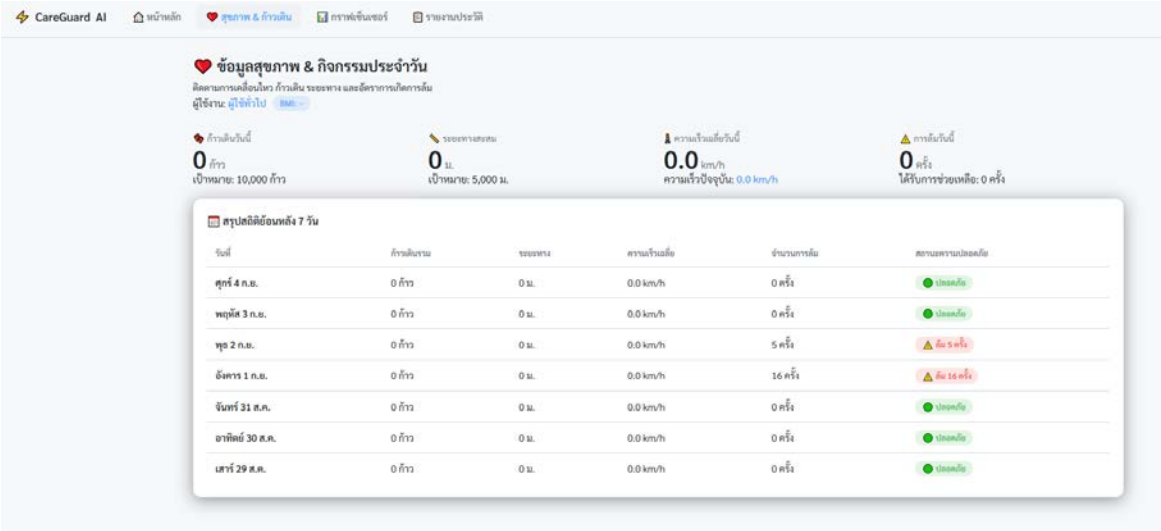
1. **การแจ้งเตือนผ่าน Telegram & Email:**

- ผู้ดูแลจะได้รับข้อความแจ้งเตือนด่วนผ่าน **Telegram Bot** ระบุรายละเอียด:
    - ชื่อผู้สูงอายุ
    - ระดับความมั่นใจของ AI (Confidence Score เช่น 98.5%)
    - ทิศทางการล้ม (เช่น ล้มไปข้างหน้า / ล้มไปข้างหลัง)
    - ตำแหน่งห้องที่เกิดเหตุ (คำนวณจากเสา RSSI ภายในบ้าน เช่น ห้องน้ำ, ห้องนอน)
    - ปุ่มลัดโทรออกหาผู้สูงอายุหรือเรียกรถพยาบาลฉุกเฉิน 1669

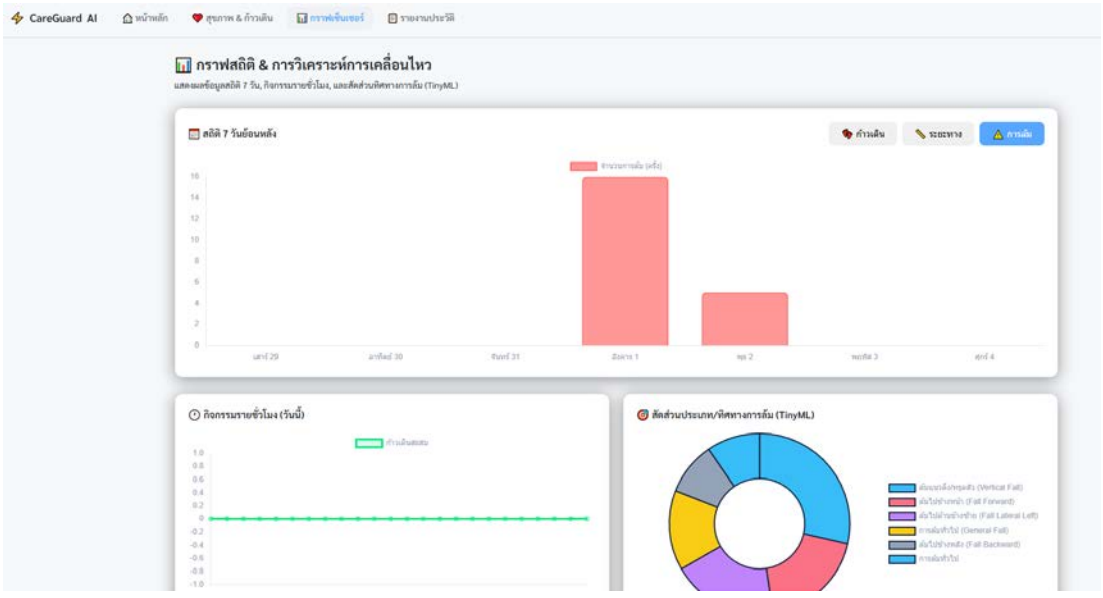
2. **การดูข้อมูลผ่าน Web Dashboard (หน้าจอควบคุม):**
    - ผู้ดูแลสามารถเปิดเบราว์เซอร์ดูหน้าจอ Dashboard ได้แบบเรียลไทม์:
        - **สถานะปัจจุบัน:** ดูว่าผู้สูงอายุกำลังทำกิจกรรมอะไร อยู่ห้องไหน
        - **ปุ่มยกเลิกสัญญาณเตือน (Cancel False Alarm):** หากผู้สูงอายุล้มแต่ไม่บาดเจ็บ หรือเป็นการลุกนั่งเร็ว สามารถกดยกเลิกสถานะฉุกเฉินได้
        - **สรุปวิเคราะห์ทางการแพทย์:** มีระบบ AI (Google Gemini) ช่วยประเมินและสรุปรายงานพฤติกรรมการเคลื่อนไหวให้ผู้ดูแลเข้าใจง่าย



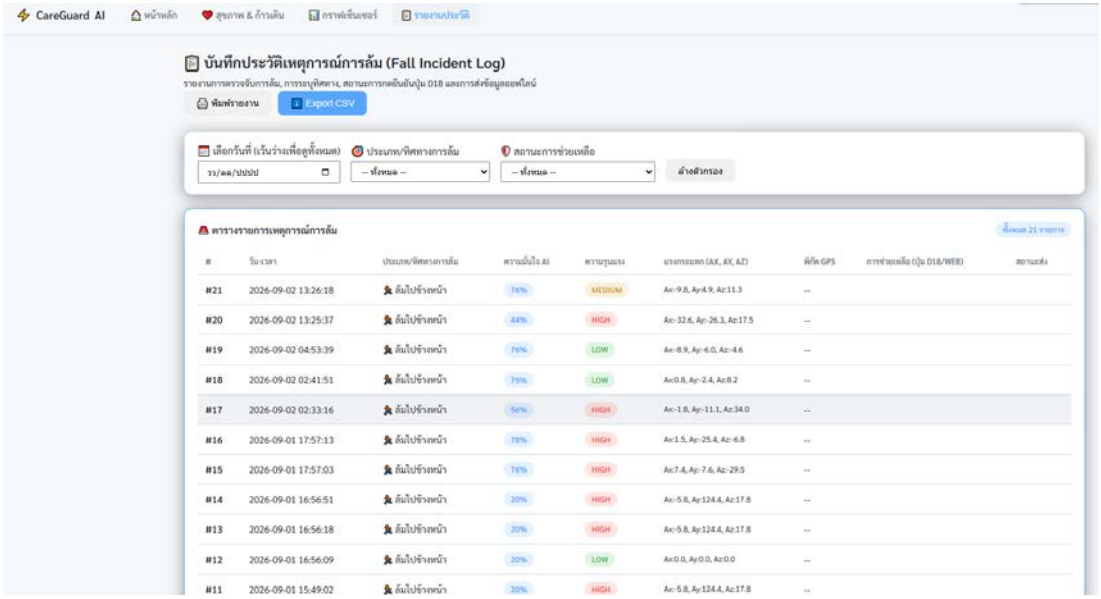
รูปที่ จ.8 **Web Dashboard** หน้าหลัก



รูปที่ จ.9 **Web Dashboard** หน้าข้อมูลสุขภาพ & กิจกรรมประจำวัน



รูปที่ จ.10 **Web Dashboard** หน้า**กราฟสถิติ & การวิเคราะห์การเคลื่อนไหว**



รูปที่ จ.11 **Web Dashboard** หน้า**บันทึกประวัติเหตุการณ์การล้ม (Fall Incident Log)**

ประวัติผู้เขียน

ประวัติผู้เขียน



ชื่อ-นามสกุล	นายธนภัทร เลิศพิทักษ์สิทธิ์
วัน-เดือน-ปีเกิด	11 มีนาคม 2546
ที่อยู่	206 หมู่ที่ 13 ต.หนองใหญ่ อ.โพนทอง
	จ.ร้อยเอ็ด 45110
สถานที่ทำงานปัจจุบัน	อาคาร 3 มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตขอนแก่น 150 ถ.ศรีจันทร์ ต.ในเมือง อ.เมือง จ.ขอนแก่น 40000

ประวัติการศึกษา

ปวส.	วิทยาลัยการอาชีพโพนทอง ปีการศึกษา 2564
	สาขาวิชาช่างอิเลิกทรอนิกส์
ปวช.	วิทยาลัยการอาชีพโพนทอง ปีการศึกษา 2561
	สาขาวิชาช่างอิเล็กทรอนิกส์